人工智能与地理数据挖掘

第九讲　人工神经网络

孙伟 sunwei29@mail.sysu.edu.cn

地理科学与规划学院

中山大學
SUN YAT-SEN UNIVERSITY

2026.6

数据挖掘/机器学习的典型方法

类别	聚类/降维	回归	分类	生成
线性方法	相关性 主成分分析 因子分析	多元线性回归	Logistic回归 判别分析	
最近邻	k均值聚类	kNN		
树型	层次聚类	决策树： 回归树	决策树： 分类树	深度 森林
密度	DBSCAN			
神经网络	SOM	ANN		
核函数		SVM（SVR）	SVM（SVC）	
图论	谱聚类	概率图模型	概率图模型	GNN
随机变量	高斯混合		贝叶斯分类	GAN
模糊数	模糊C 均值聚类	模糊逻辑模型 ANFIS	模糊逻辑模型	FDNN
深度学习	自动编码器	CNN，LSTM、Transformer		

主要内容

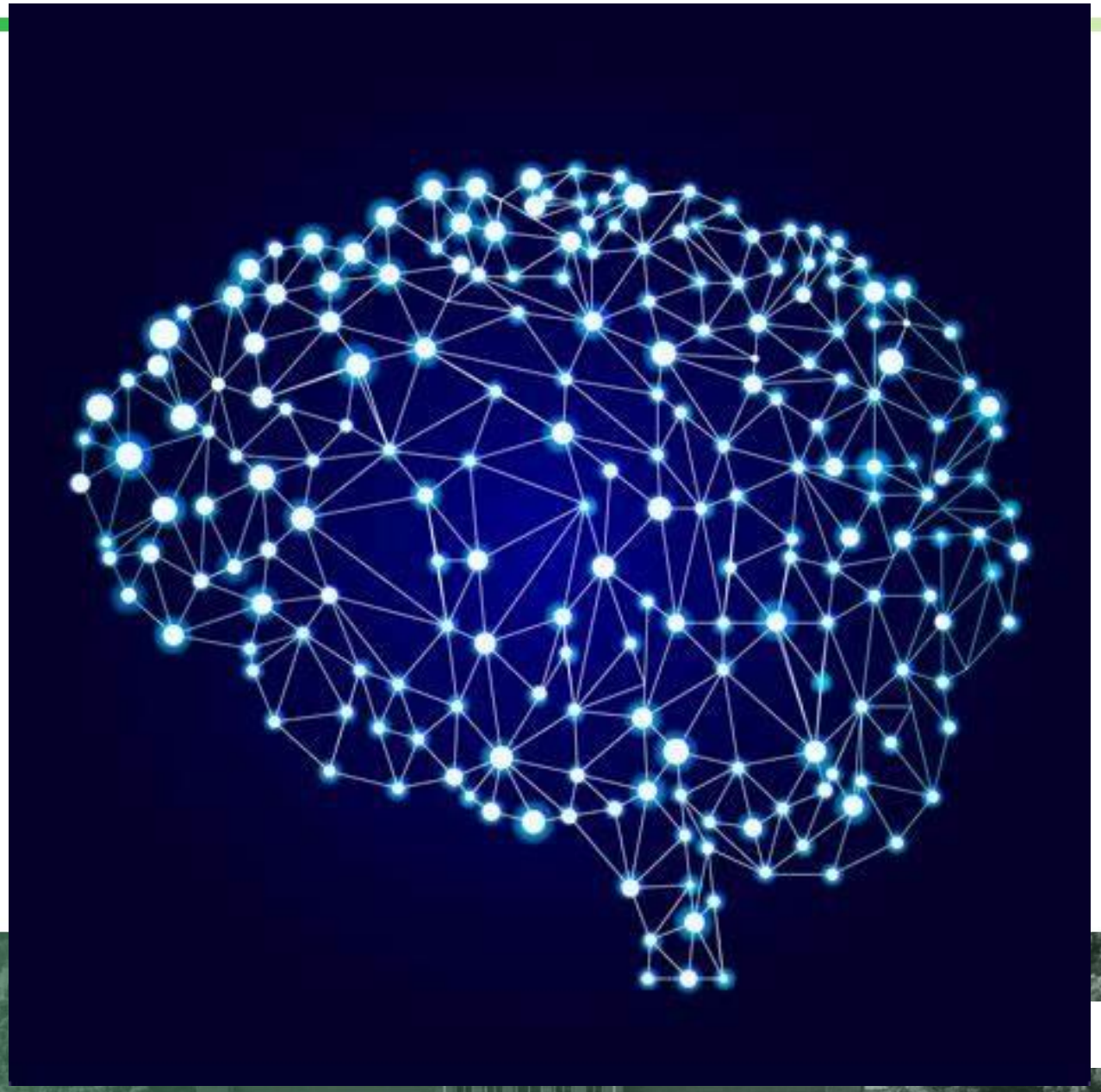
1. ANN的实例
2. ANN发展史
3. ANN的简介
4. 典型的ANN
5. ANN的求解：误差反向传播算法
6. 用Matlab构建ANN
7. 小结

ANN的实例

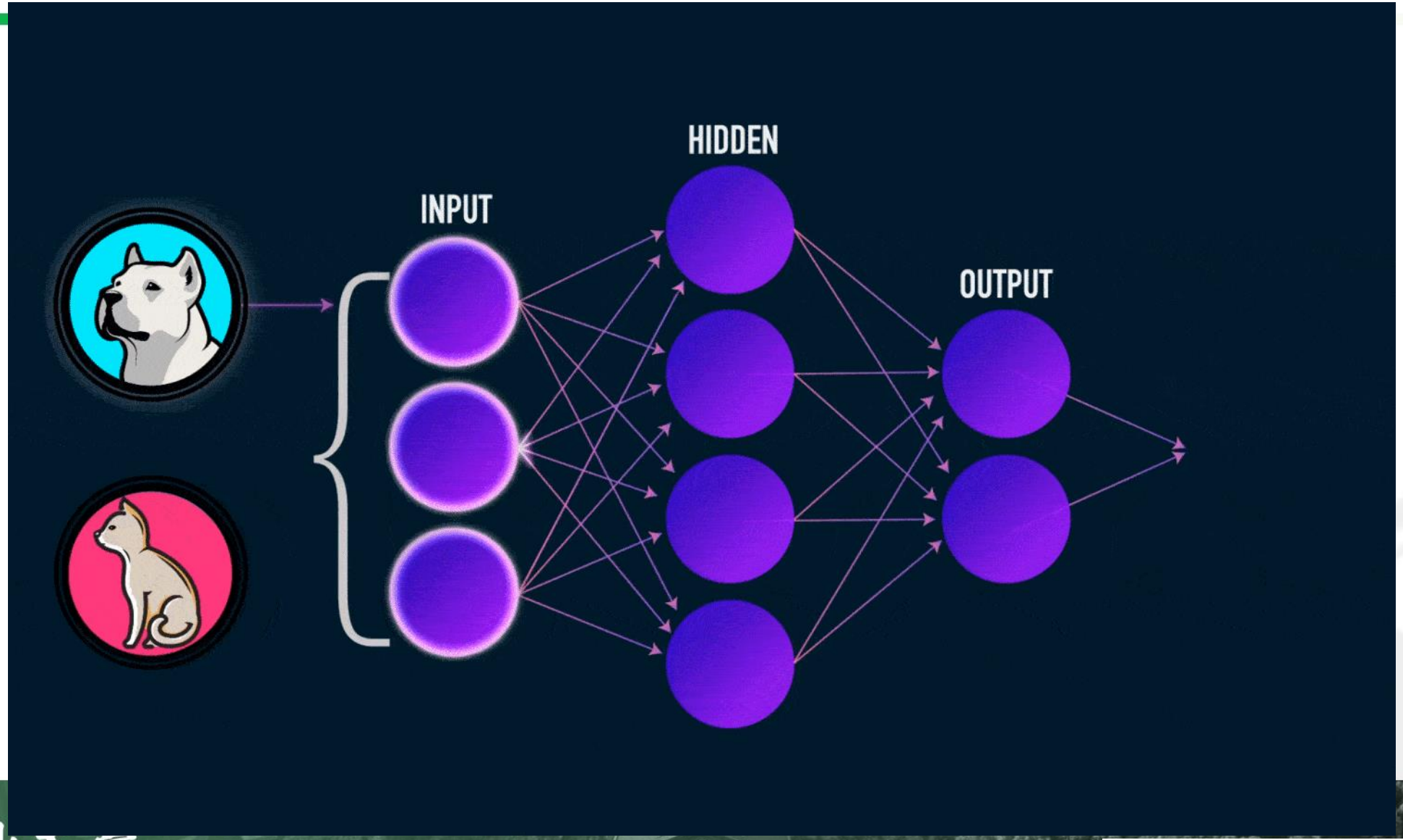
神经网络的结构



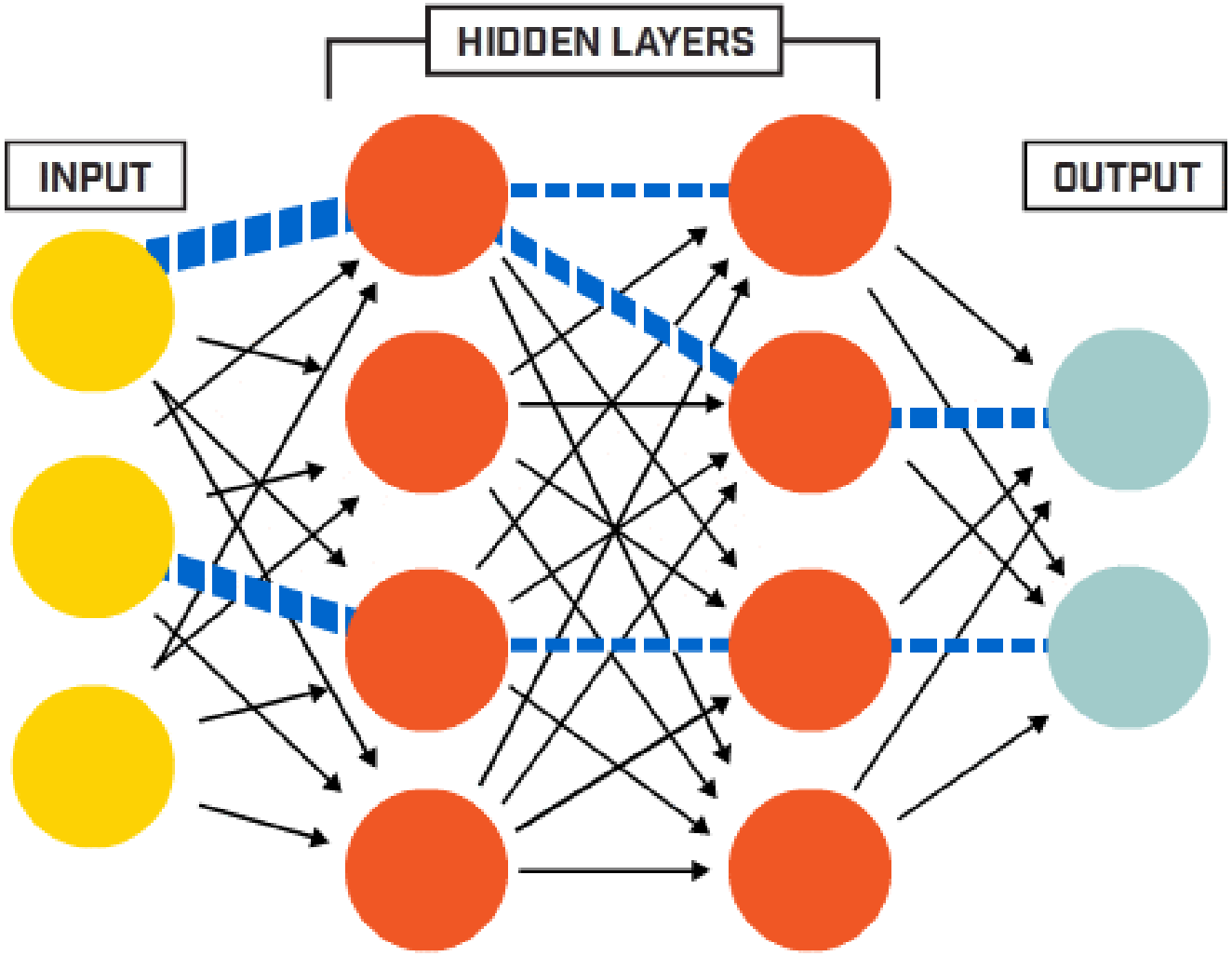
神经网络的结构



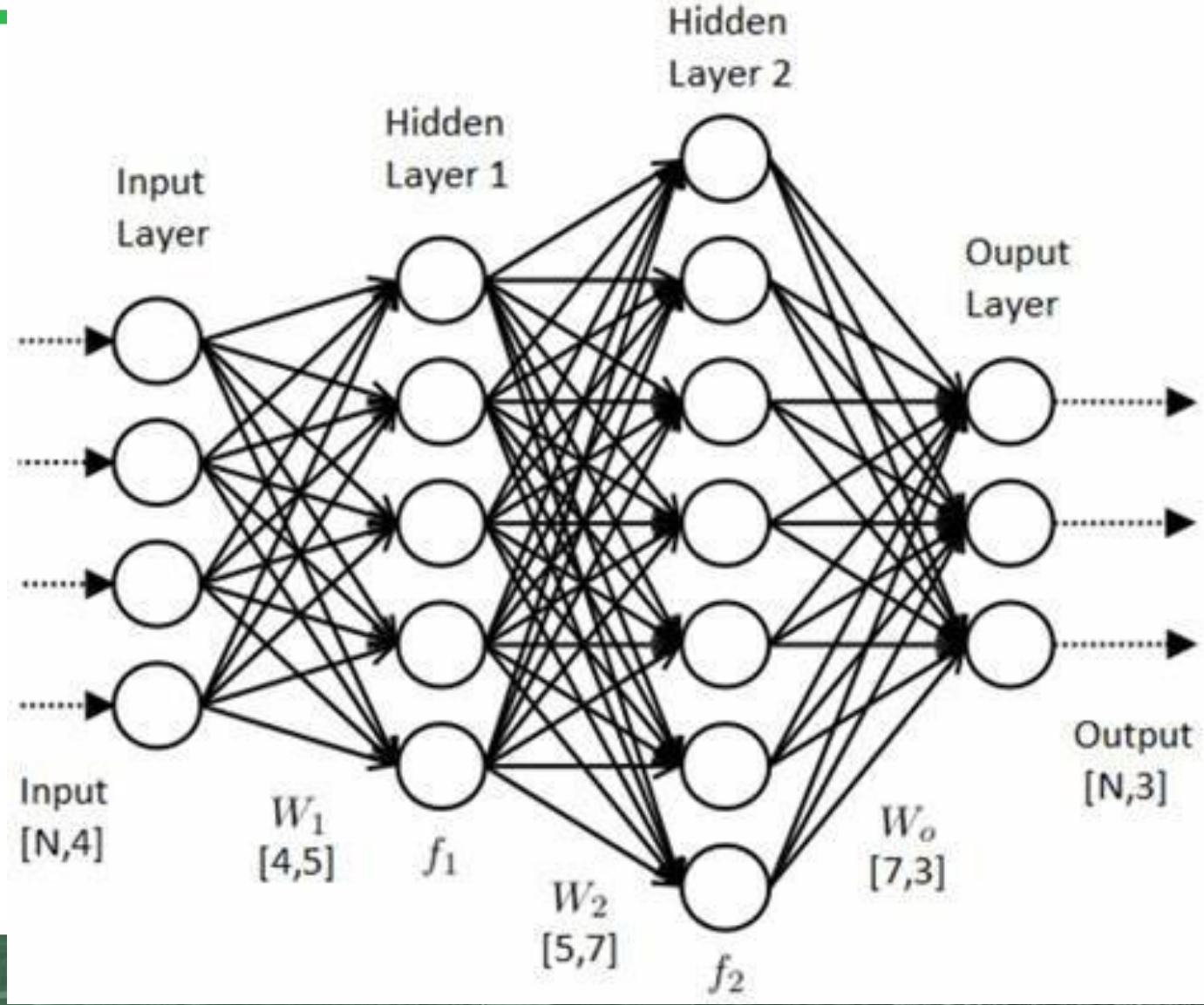
人工神经网络的结构



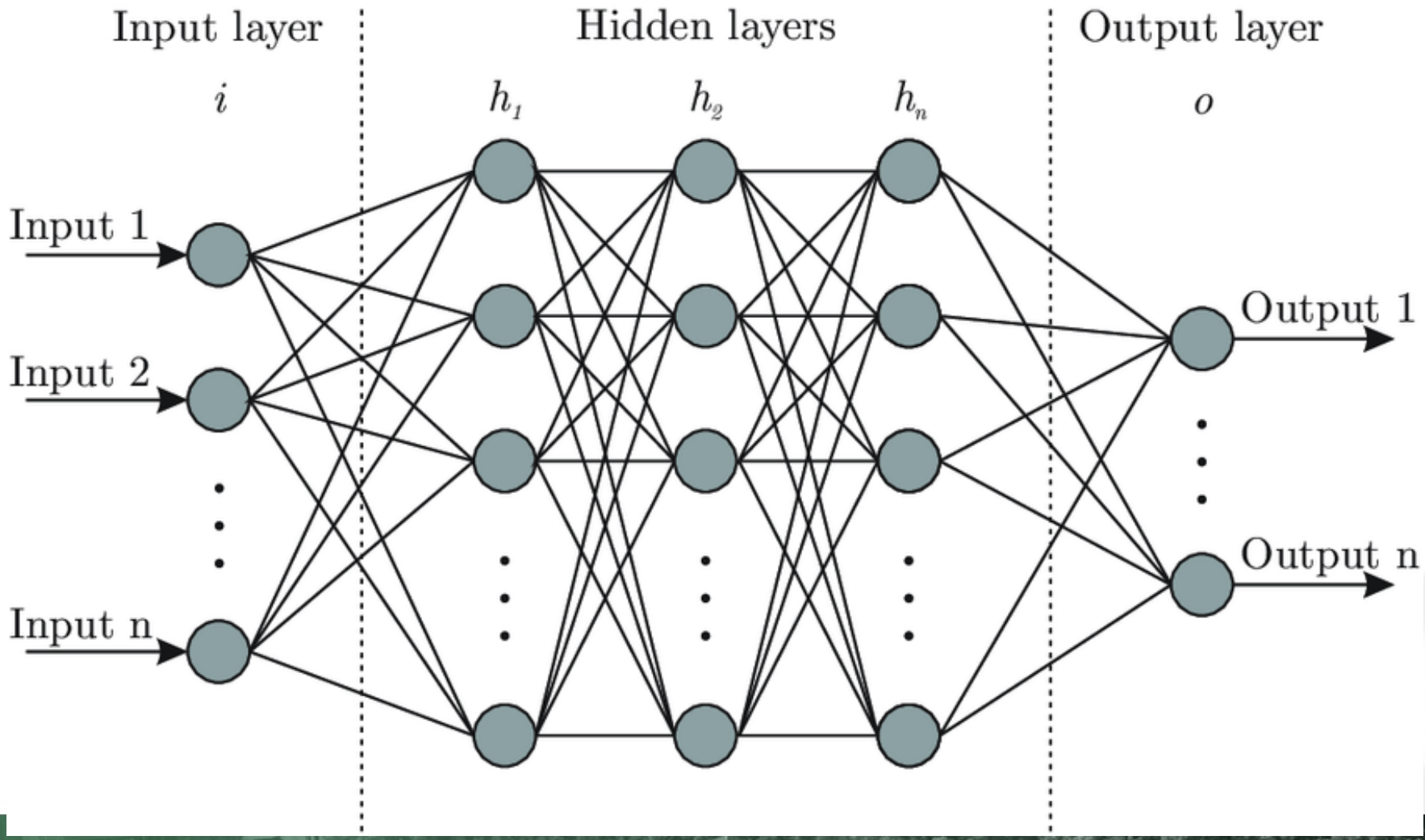
人工神经网络的结构



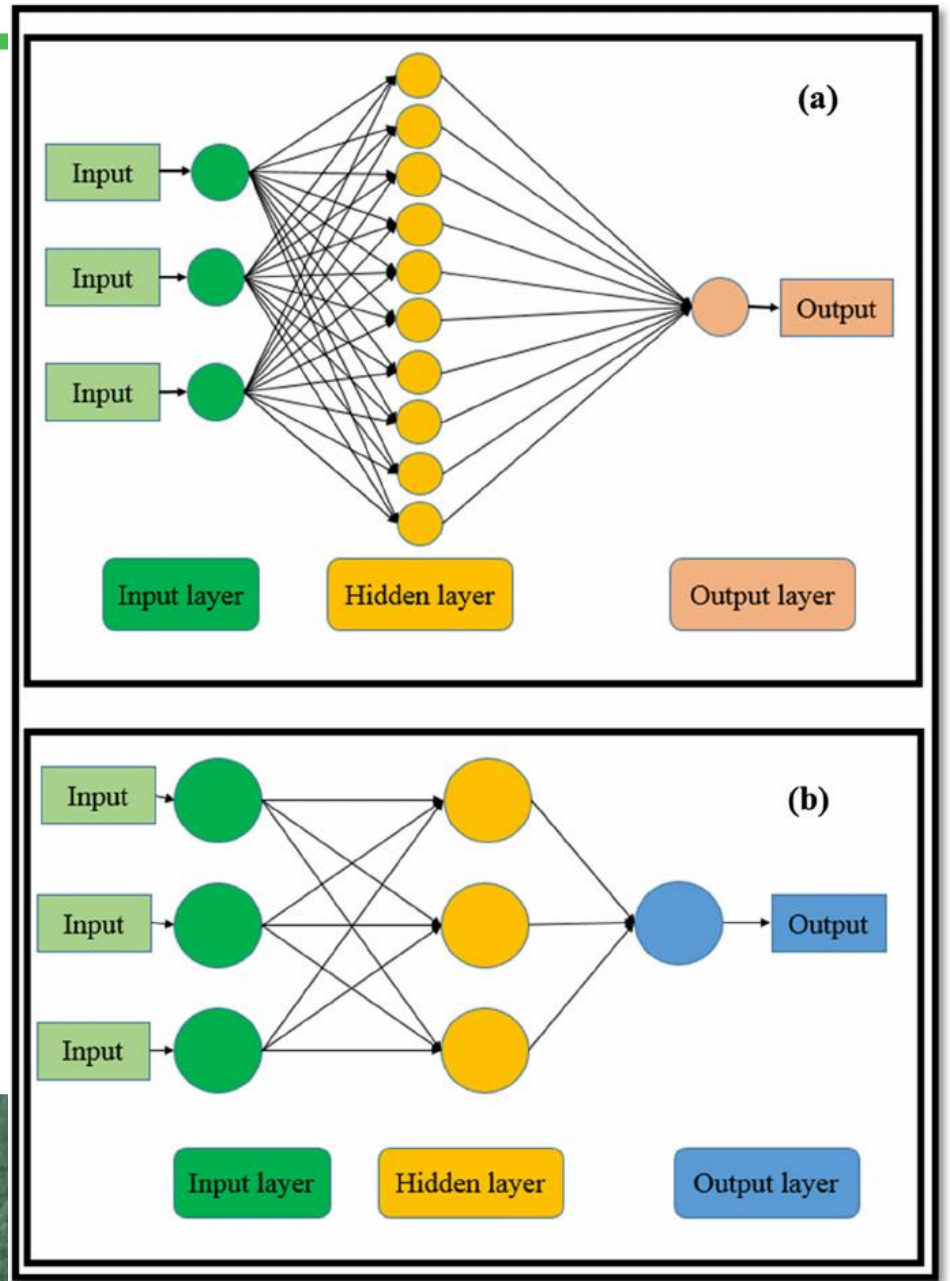
人工神经网络的结构


Hidden
Layer 2
Hidden
Layer 1
Input
Layer
Ouput
Layer
Input
[N,4]
W_1
[4,5]
f_1
W_2
[5,7]
f_2
W_o
[7,3]
Output
[N,3]

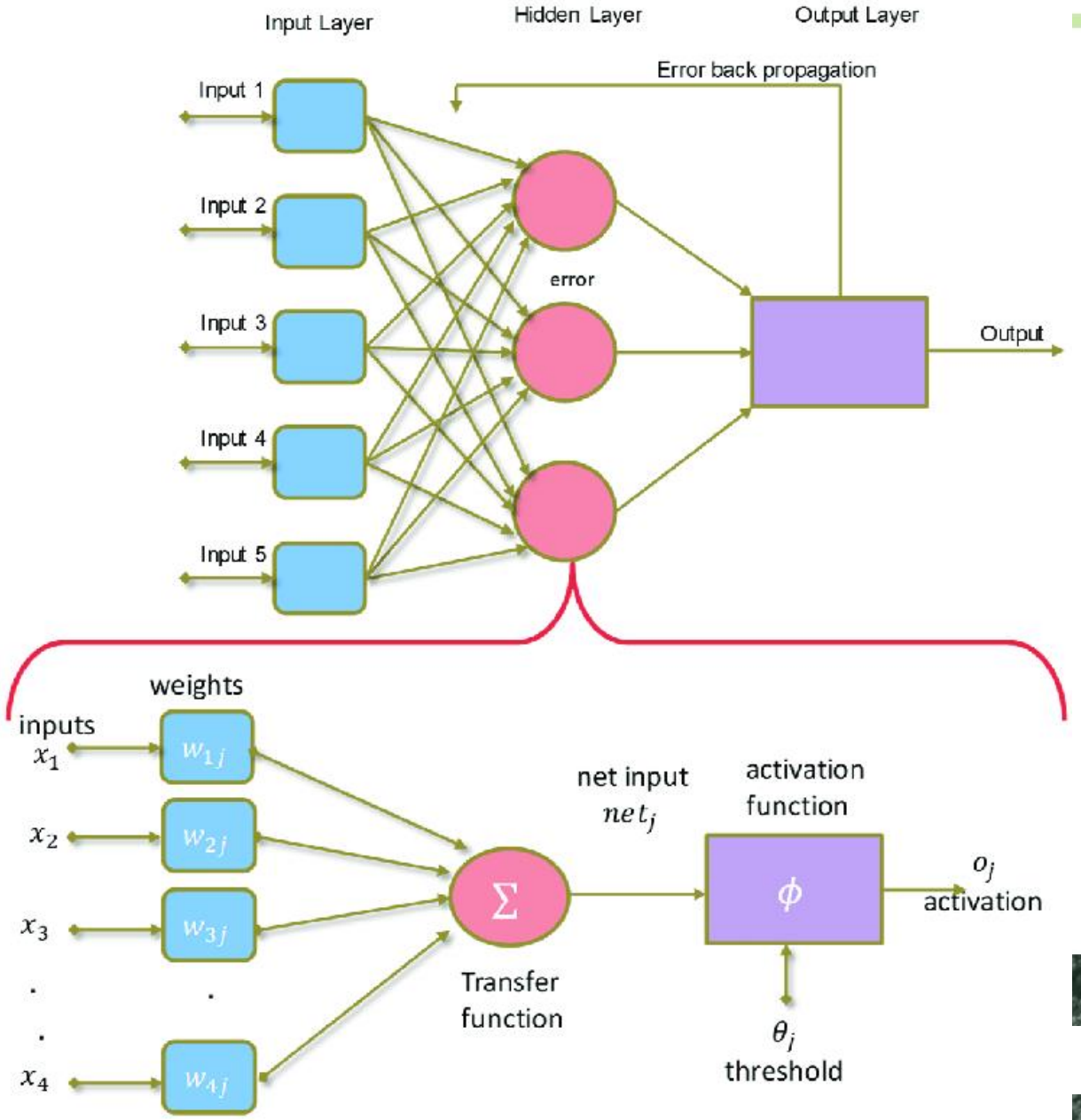
人工神经网络的结构


Input layer
Hidden layers
Output layer
i
h_1
h_2
h_n
o
Input 1
Input 2
Input n
Output 1
Output n

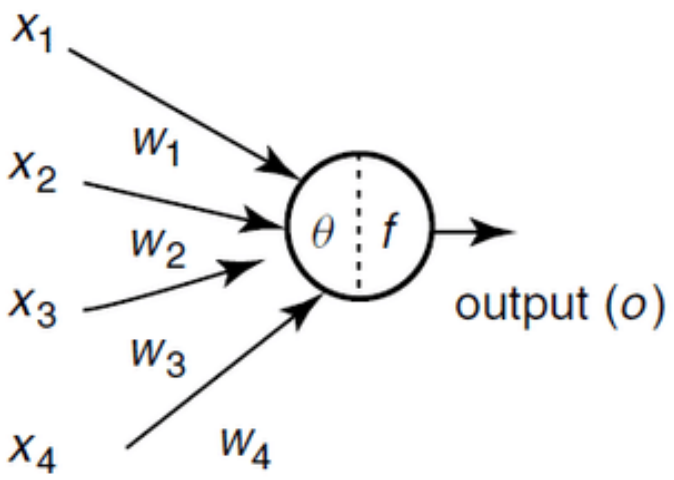
人工神经网络的结构

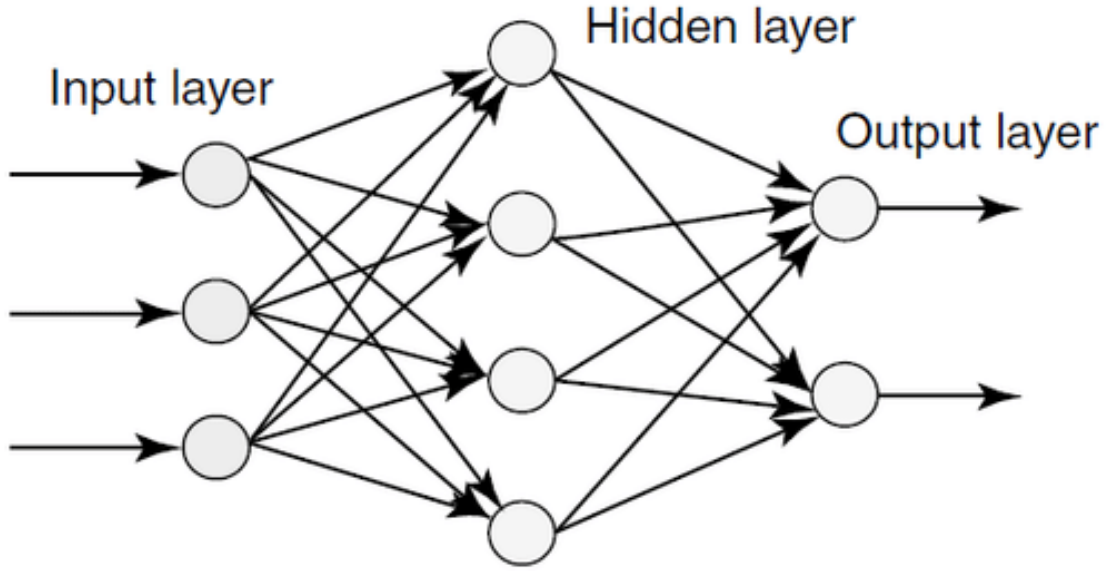

(a)
Input
Input
Input
Output
Input layer
Hidden layer
Output layer
(b)
Input
Input
Input
Output
Input layer
Hidden layer
Output layer

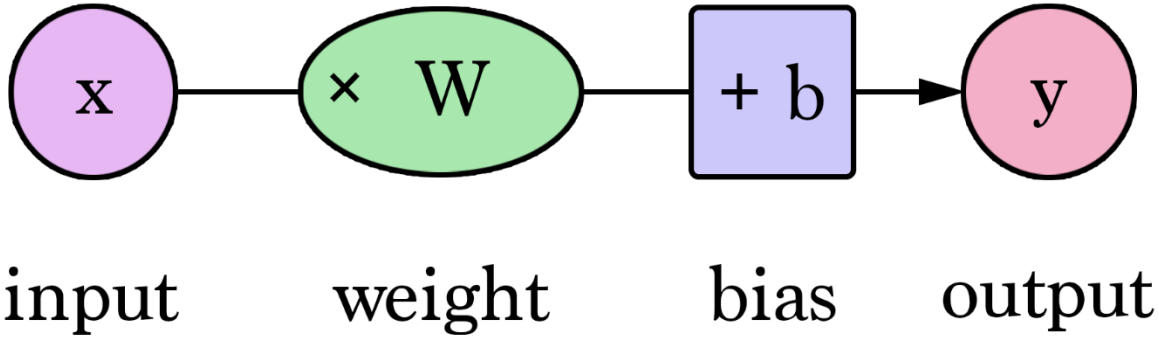
人工神经网络的结构


Input Layer
Hidden Layer
Output Layer
Error back propagation
Input 1
Input 2
Input 3
Input 4
Input 5
error
Output
weights
inputs
x_1
x_2
x_3
x_4
w_{1j}
w_{2j}
w_{3j}
w_{4j}
Σ
Transfer function
net input net_j
activation function
ϕ
o_j
activation
θ_j
threshold

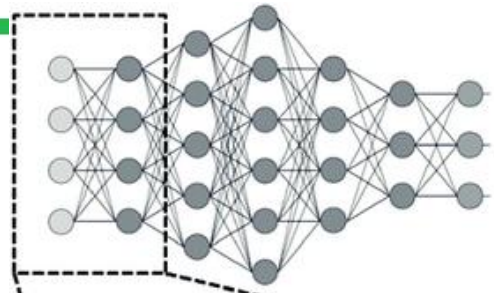
人工神经网络的结构


x_1
x_2
x_3
x_4
w_1
w_2
w_3
w_4
θ
f
output (o)


Input layer
Hidden layer
Output layer

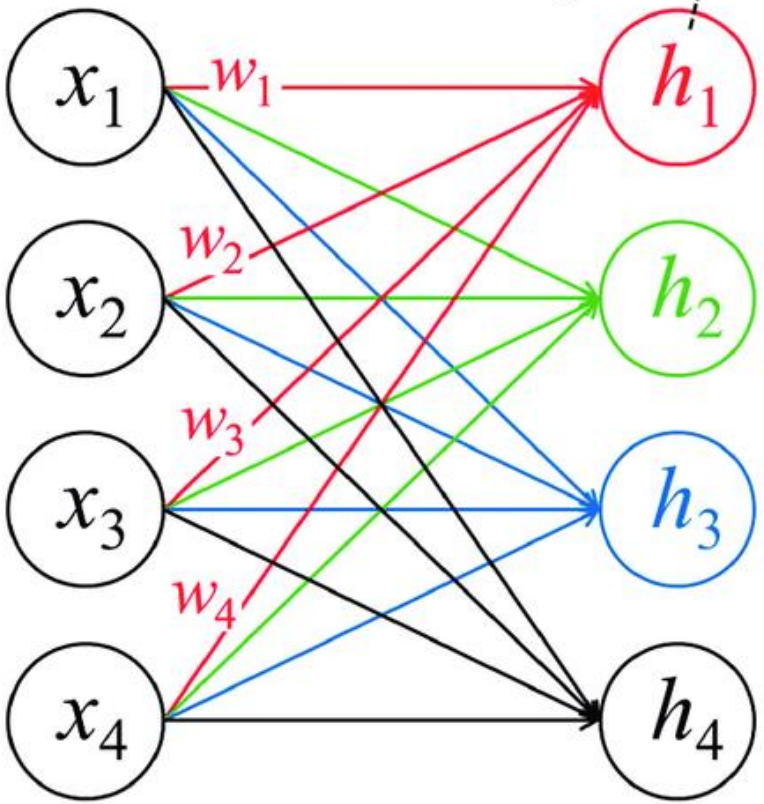

x
× W
+ b
y
input
weight
bias
output

人工神经网络的结构



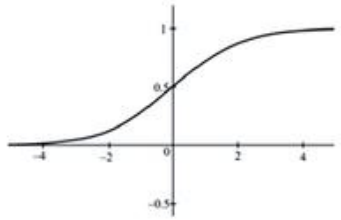
Activation function

$$h = f(\sum w_i \times x_i)$$

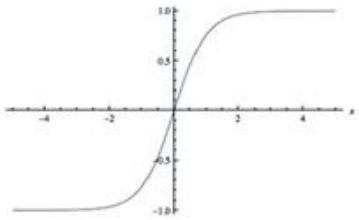


Common activation functions

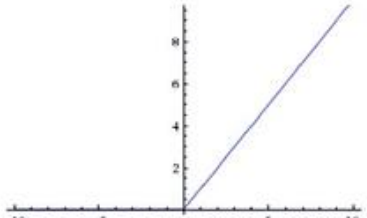
- Logistic (sigmoid)



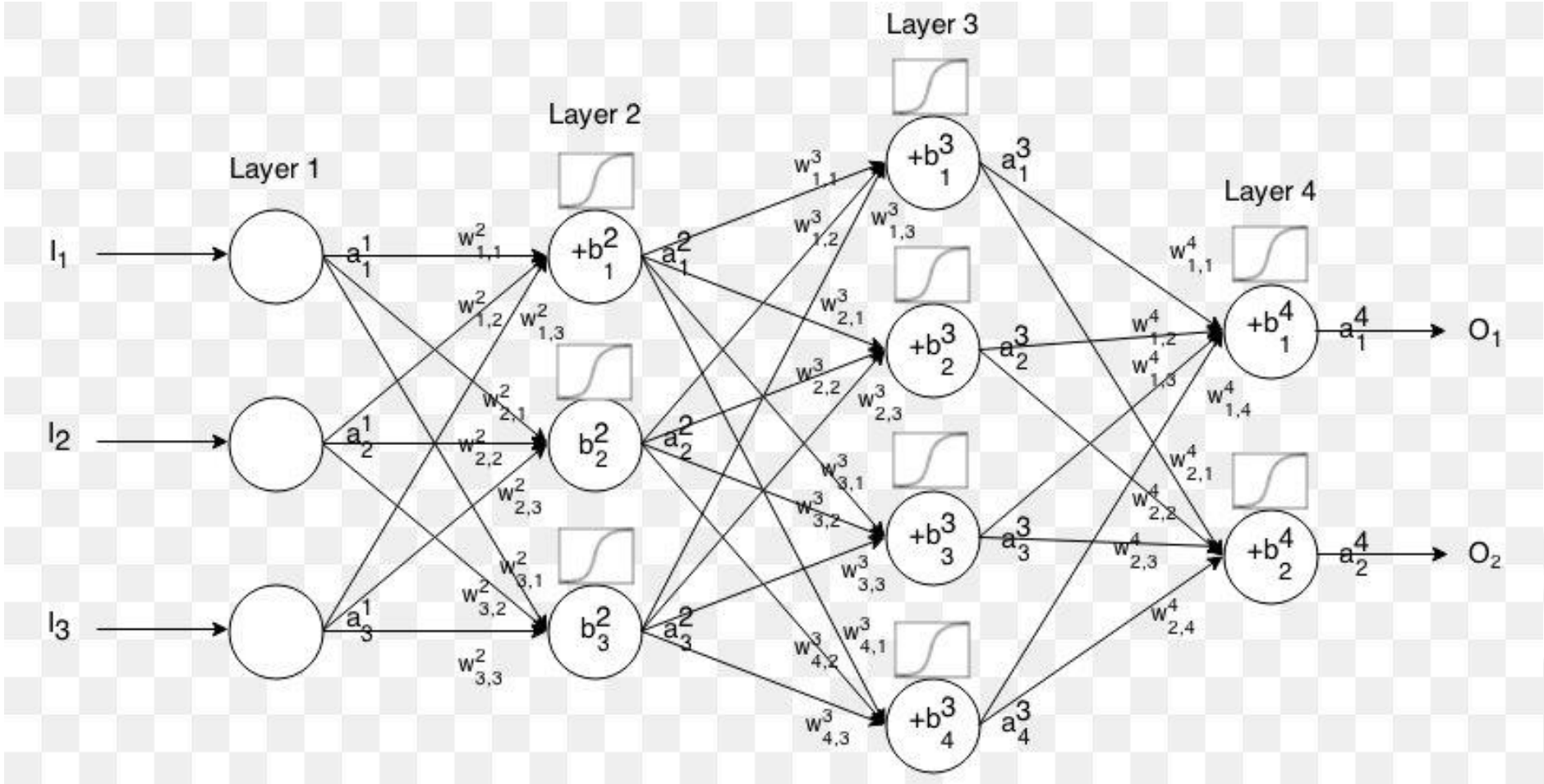
- Tanh



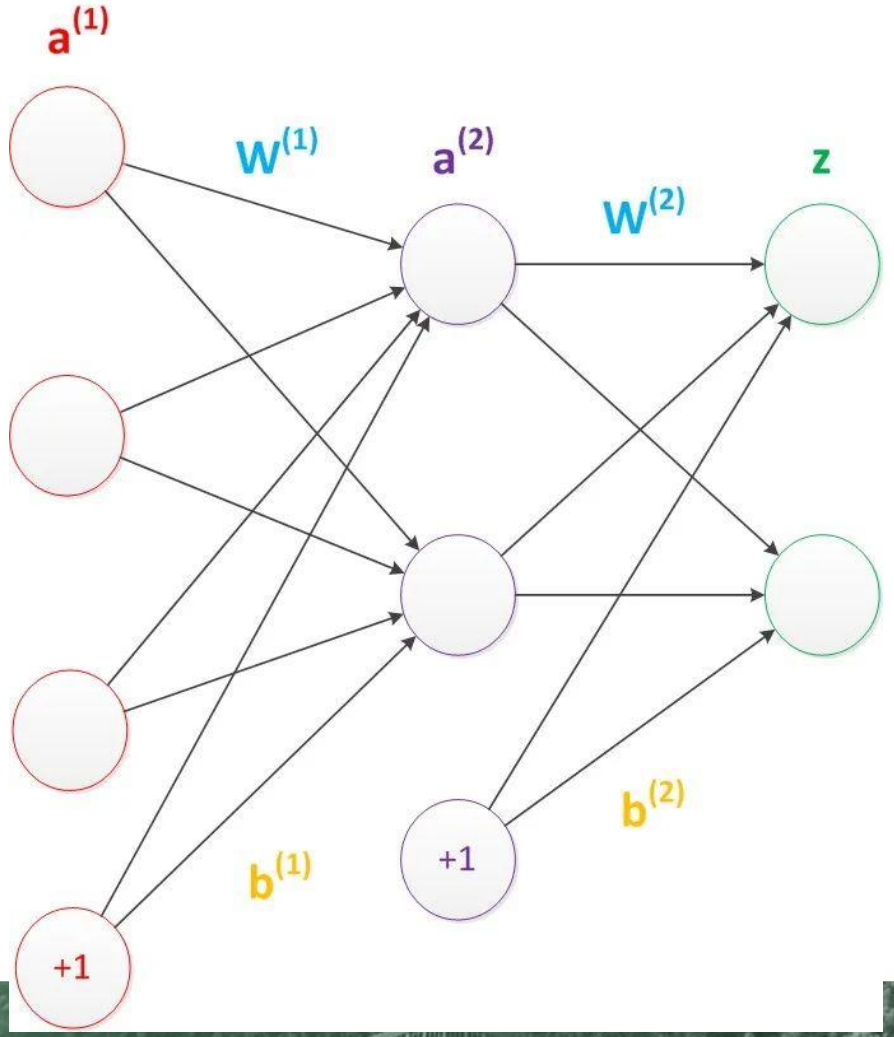
- ReLU



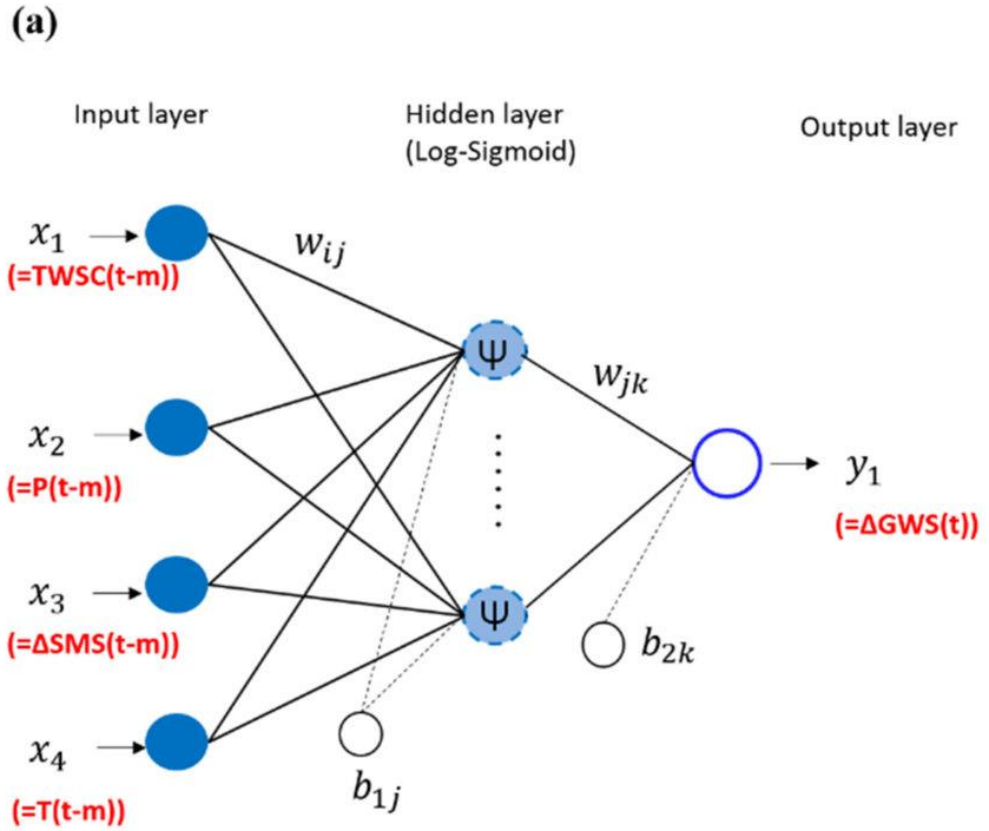
人工神经网络的结构

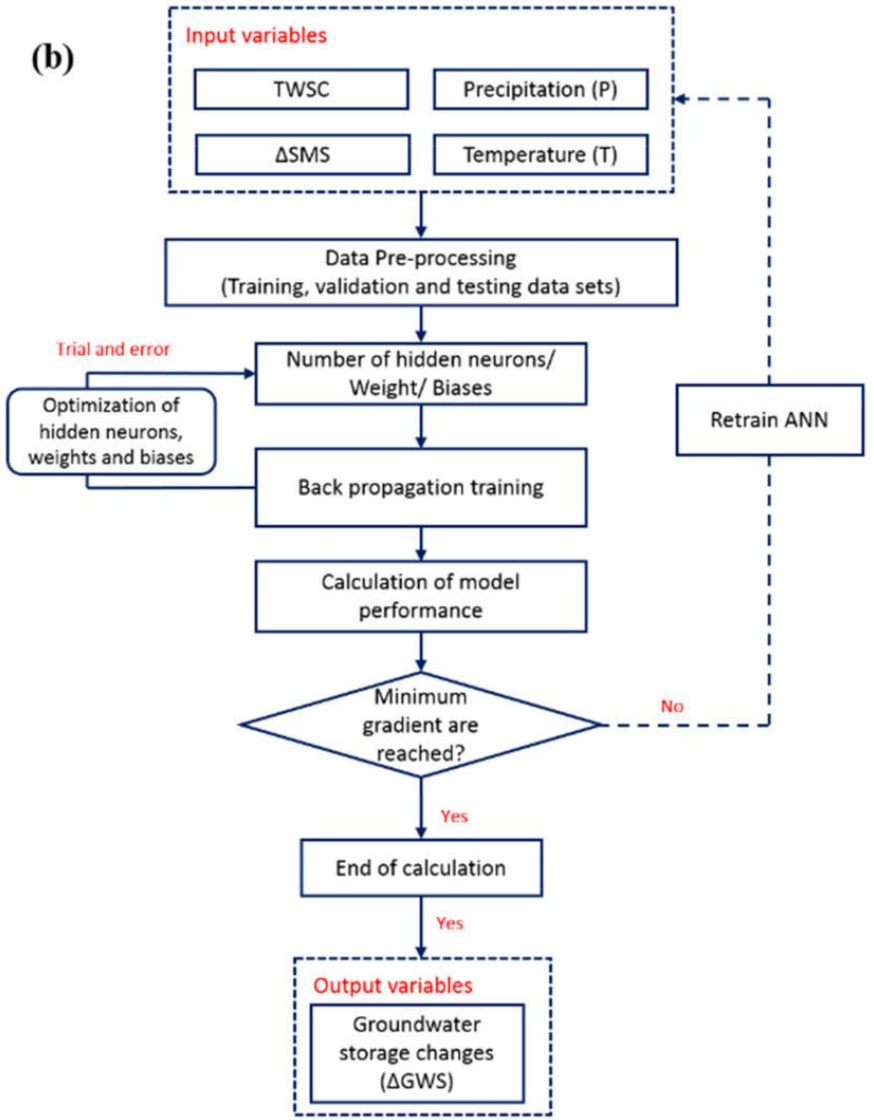


人工神经网络的结构



人工神经网络的结构

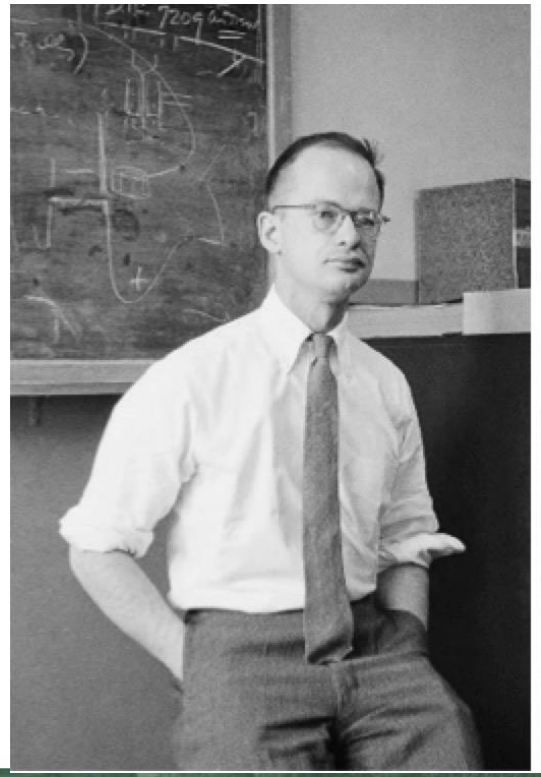

(a)
Input layer
Hidden layer
(Log-Sigmoid)
Output layer
x_1
(=TWSC(t-m))
x_2
(=P(t-m))
x_3
(=ΔSMS(t-m))
x_4
(=T(t-m))
w_{ij}
w_{jk}
b_{1j}
b_{2k}
y_1
(=ΔGWS(t))

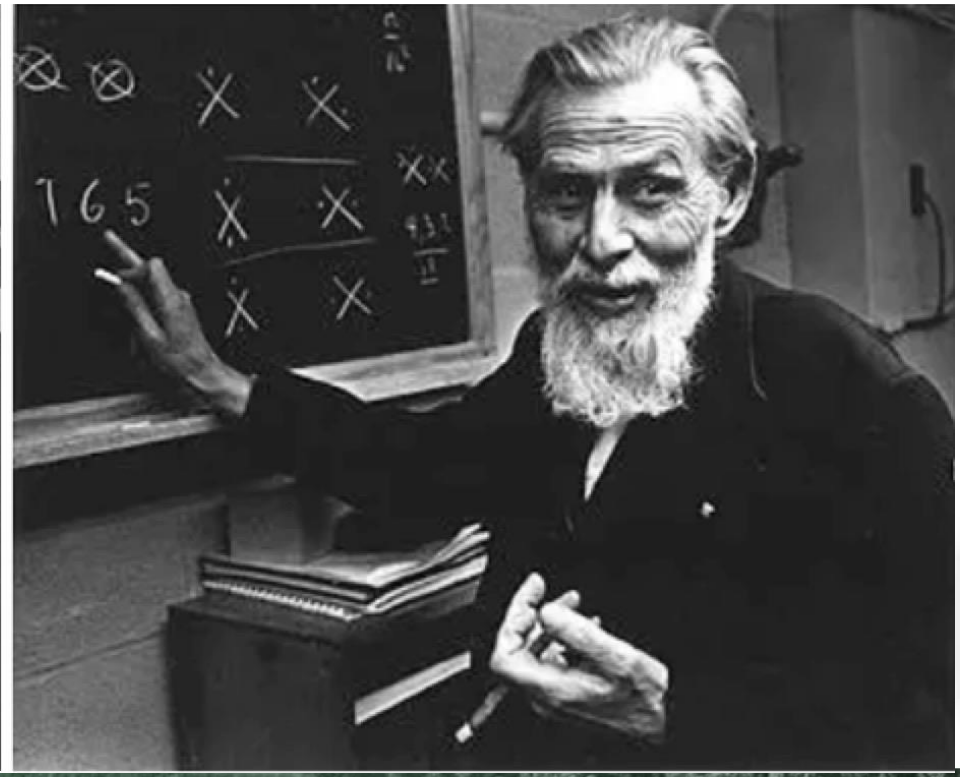

(b)
Input variables
TWSC
Precipitation (P)
ΔSMS
Temperature (T)
Data Pre-processing
(Training, validation and testing data sets)
Trial and error
Optimization of hidden neurons, weights and biases
Number of hidden neurons/ Weight/ Biases
Back propagation training
Calculation of model performance
Minimum gradient are reached?
No
Retrain ANN
Yes
End of calculation
Yes
Output variables
Groundwater storage changes (ΔGWS)

ANN发展史

神经网络发展史：第一阶段

1943年, McCulloch和Pitts 提出第一个神经元数学模型, 即M-P模型, 并从原理上证明了人工神经网络能够计算任何算数和逻辑函数



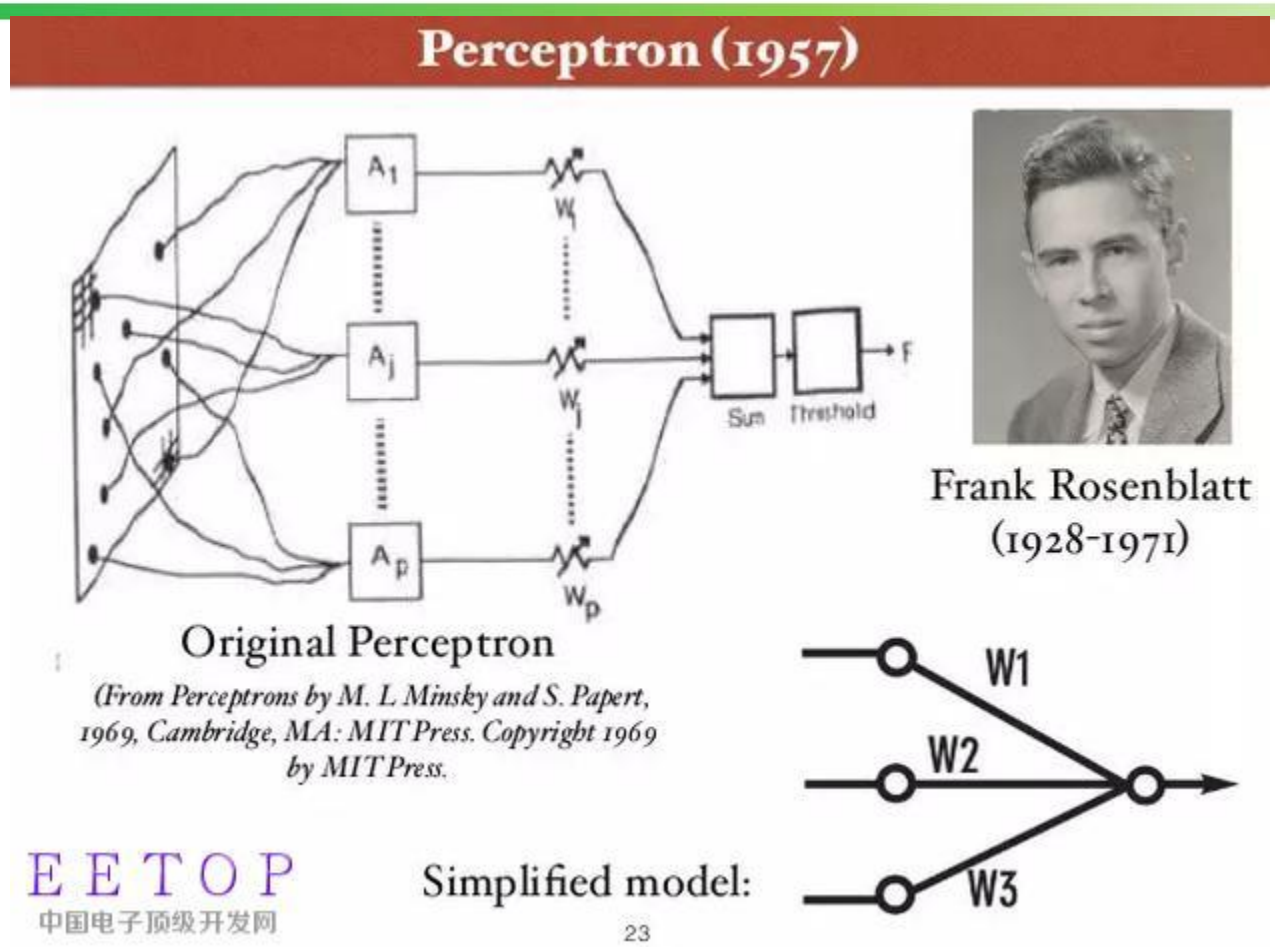


神经网络发展史：第一阶段



1949年, Hebb 发表《The Organization of Behavior》一书, 提出生物神经元学习的机理, 即Hebb学习规则：神经网络的学习过程最终是发生在神经元之间的突触部位，突触的联结强度随着突触前后神经元的活动而变化，变化的量与两个神经元的活性之和成正比。

神经网络发展史：第一阶段



EETOP
中国电子顶级开发网

- 1958年, Rosenblatt 提出**感知机网络（Perceptron）模型和其学习规则**

神经网络发展史：第一阶段

- 1960年, Widrow和Hoff提出**自适应线性神经元**（Adaline）模型和**最小均方学习算法**
- 1969年, Minsky和Papert 发表《Perceptrons》一书, 指出**单层神经网路不能解决非线性问题, 多层网络的训练算法尚无希望**. 这个论断导致神经网络进入低谷

神经网络发展史：第二阶段



1982年，物理学家Hopfield提出了一种具有联想记忆、优化计算能力的递归网络模型，即Hopfield 网络

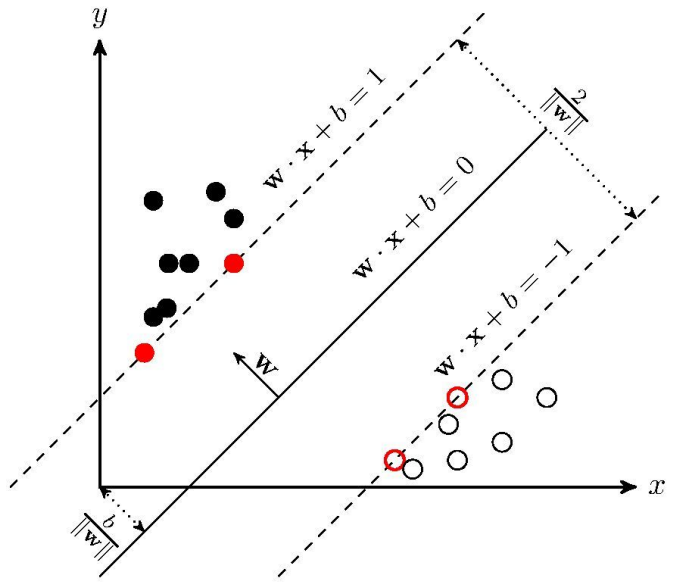
神经网络发展史：第二阶段



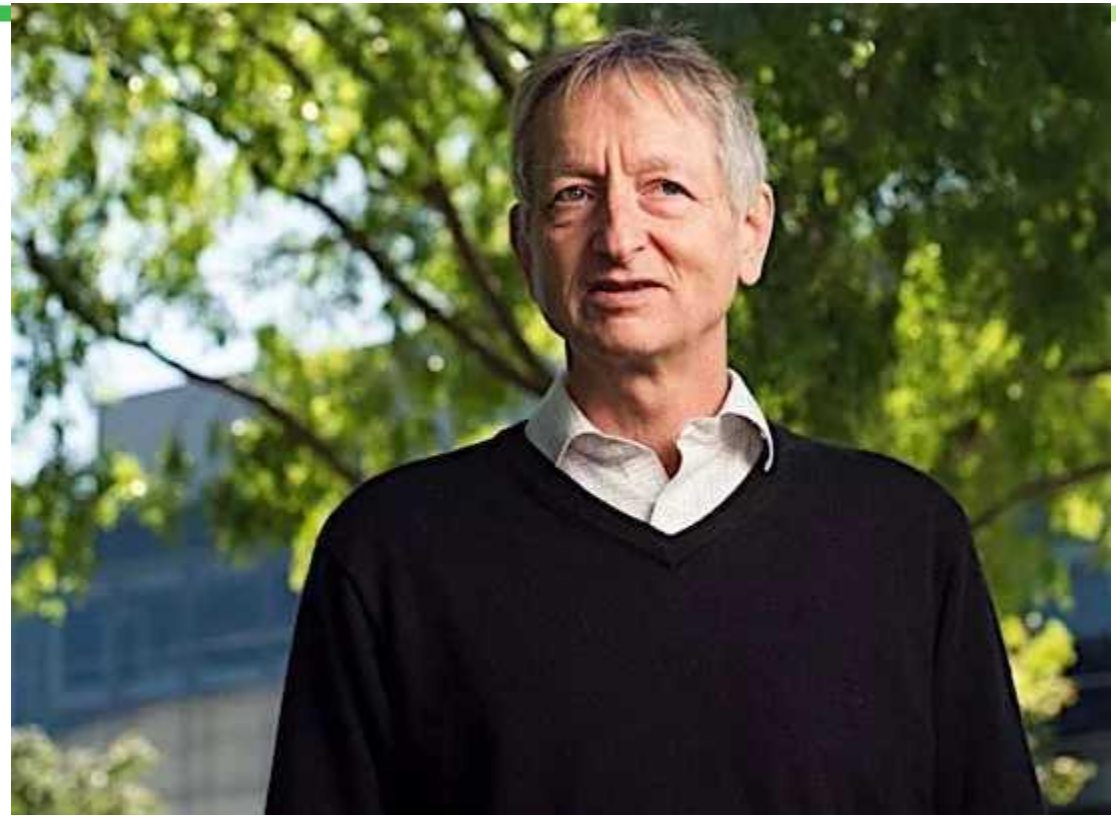
1986年, Rumelhart 等编辑的著作《Parallel Distributed Proceesing:Explorations in the Microstructures of Cognition》报告了反向传播算法

神经网络发展史：第二阶段

- 1987年, IEEE 在美国加州圣地亚哥召开第一届神经网络国际会议（ICNN）
- 90年代初, 伴随**统计学习理论和SVM**的兴起, 神经网络由于理论不够清楚, 试错性强, 难以训练, 再次进入低谷



神经网络发展史：第三阶段



- 2006年, Hinton提出**深度信念网络(DBN)**, 通过“预训练+微调”使得深度模型的最优化变得相对容易
- 2012年, Hinton 组参加ImageNet 竞赛, 使用 **CNN模型**以超过第二名10个百分点成绩夺得当年竞赛冠军

神经网络发展史：第三阶段

- 伴随**云计算、大数据时代**的到来，计算能力的大幅提升，使得深度学习模型在**计算机视觉、自然语言处理、语音识别**等众多领域都取得了较大的成功

Images & Video





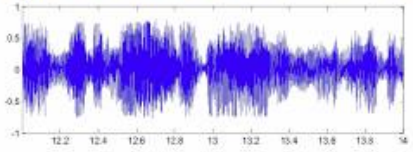
Text & Language

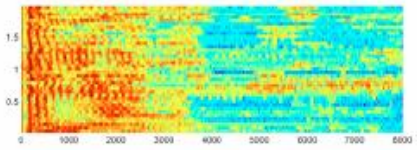


REUTERS

Associated Press

Speech & Audio



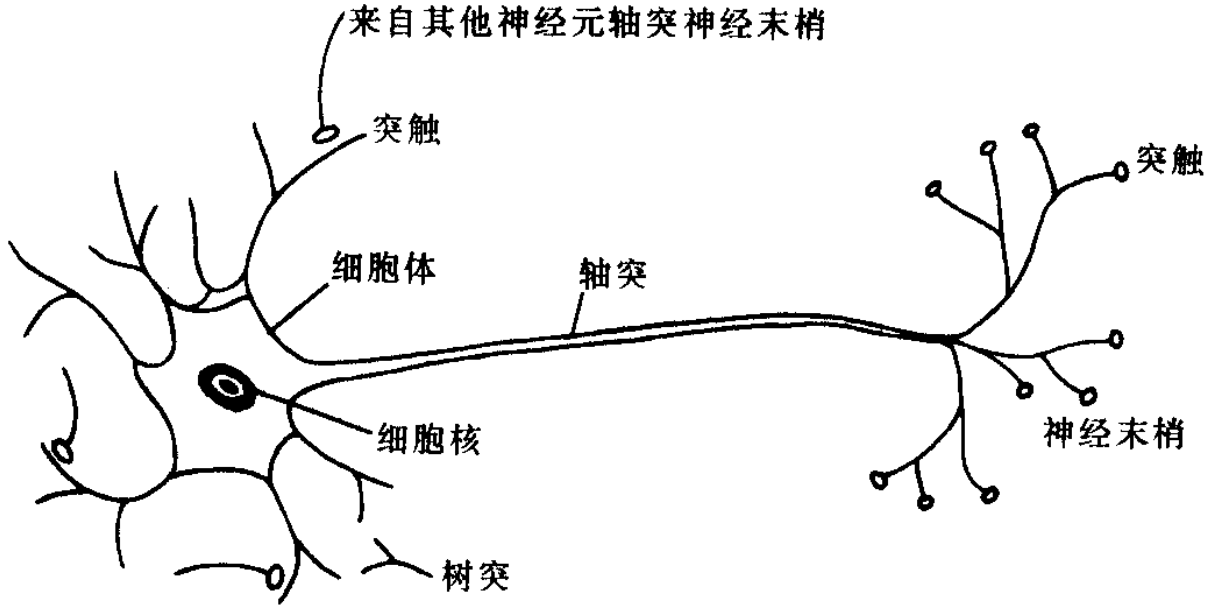




ANN的定义

生物神经网络的定义

- 多个神经元组成；每个神经元与其他神经元相连，
- 当某神经元“兴奋”时，向相连的神经云发送化学物质，从而改变这些神经元内的电位；
- 如果某神经元的电位超过“阈值”，就会被激活（“兴奋”起来），向其它神经元发送化学物质。



人工神经网络的定义

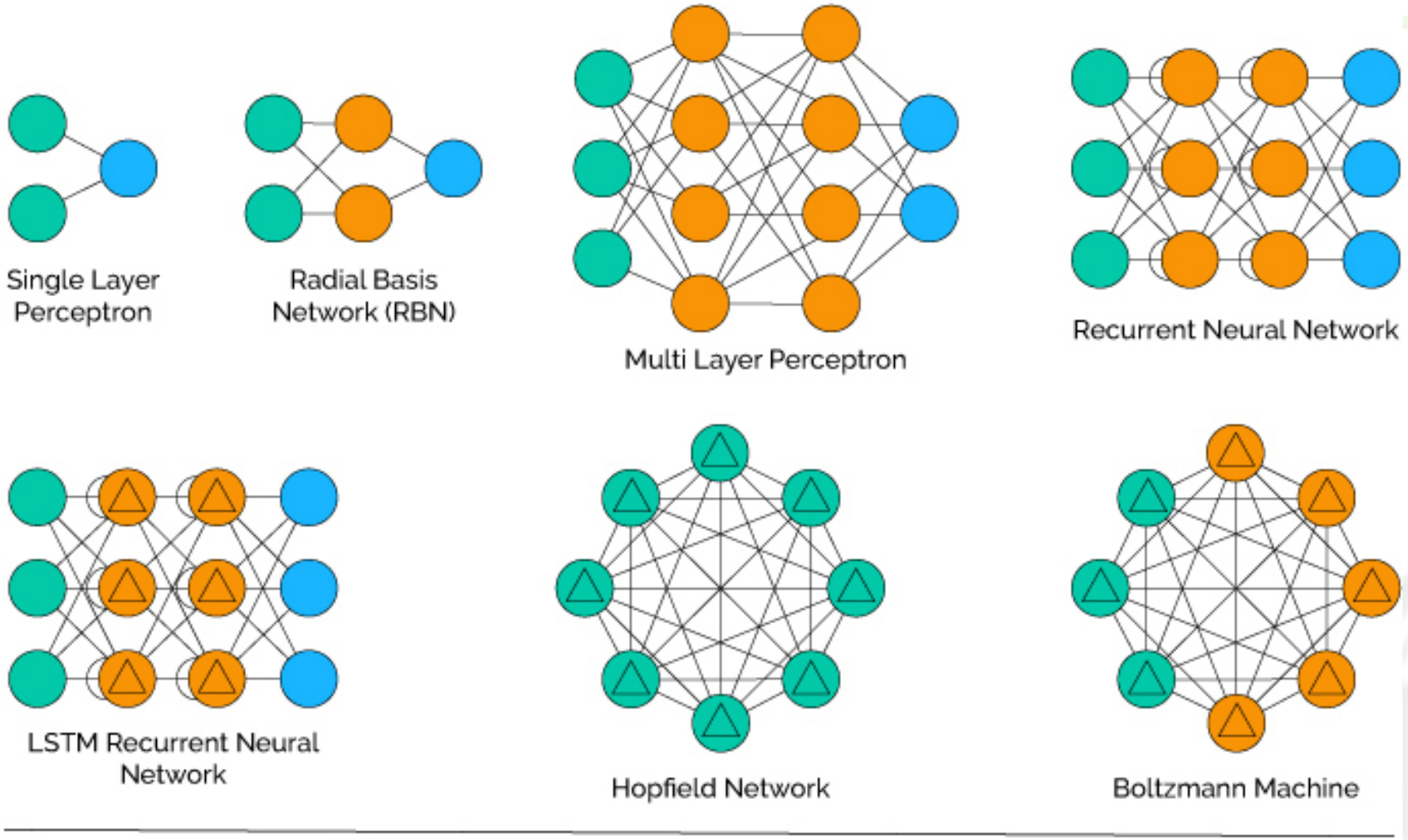
”人工神经网络”是由具有适应性的简单单元组成的广泛并行互联的网络，它的组织能够模拟生物神经系统对真实世界物体所作出的反应”

。 [Kohonen, 1988]

- 机器学习中的神经网络通常是指“神经网络学习” 或者机器学习与神经网络两个学科的交叉部分。
- 神经元模型即上述定义中的“简单单元”，是神经网络的基本成分

。

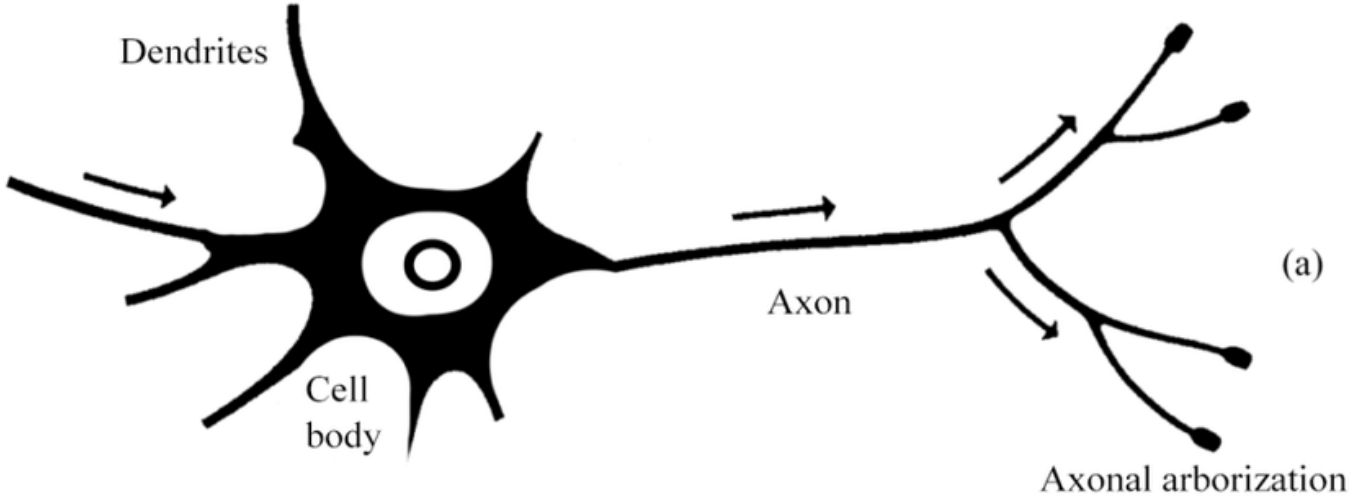
人工神经网络的结构

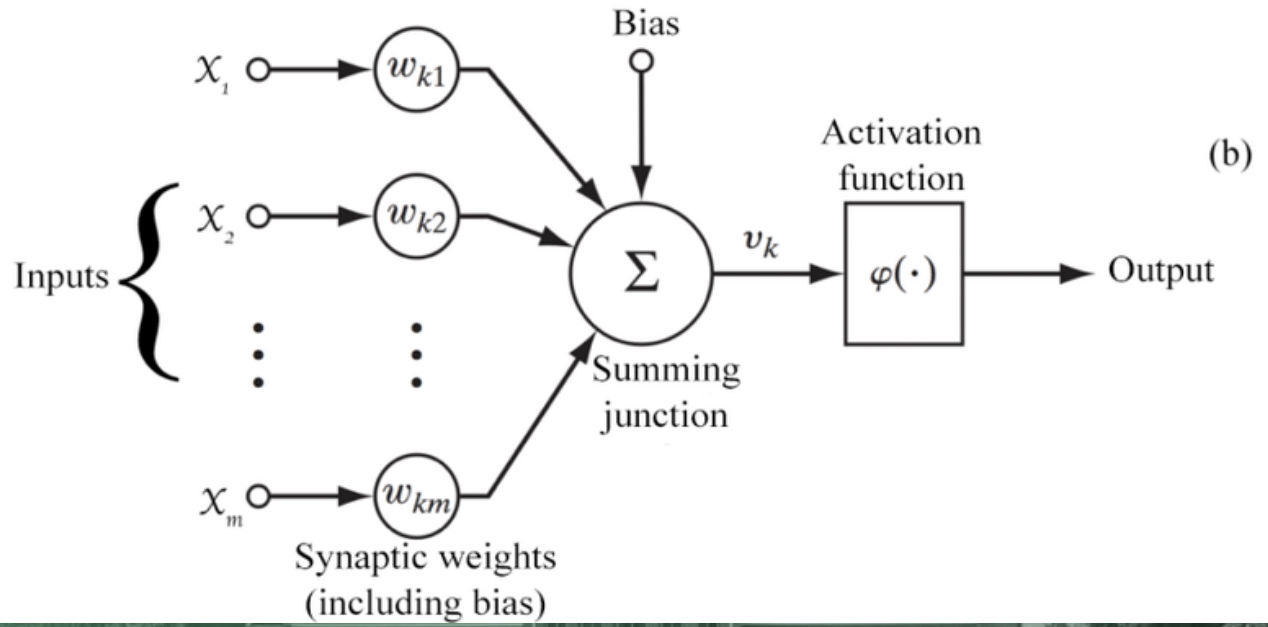

Single Layer Perceptron
Radial Basis Network (RBN)
Multi Layer Perceptron
Recurrent Neural Network
LSTM Recurrent Neural Network
Hopfield Network
Boltzmann Machine
Input Unit
Hidden Unit
Backfed Input Unit
Output Unit
Feedback with Memory Unit
Probabilistic Hidden Unit

3. 不同类型的ANN

- 神经元模型
- 激活函数
- 感知机模型
- 多层前馈神经网络

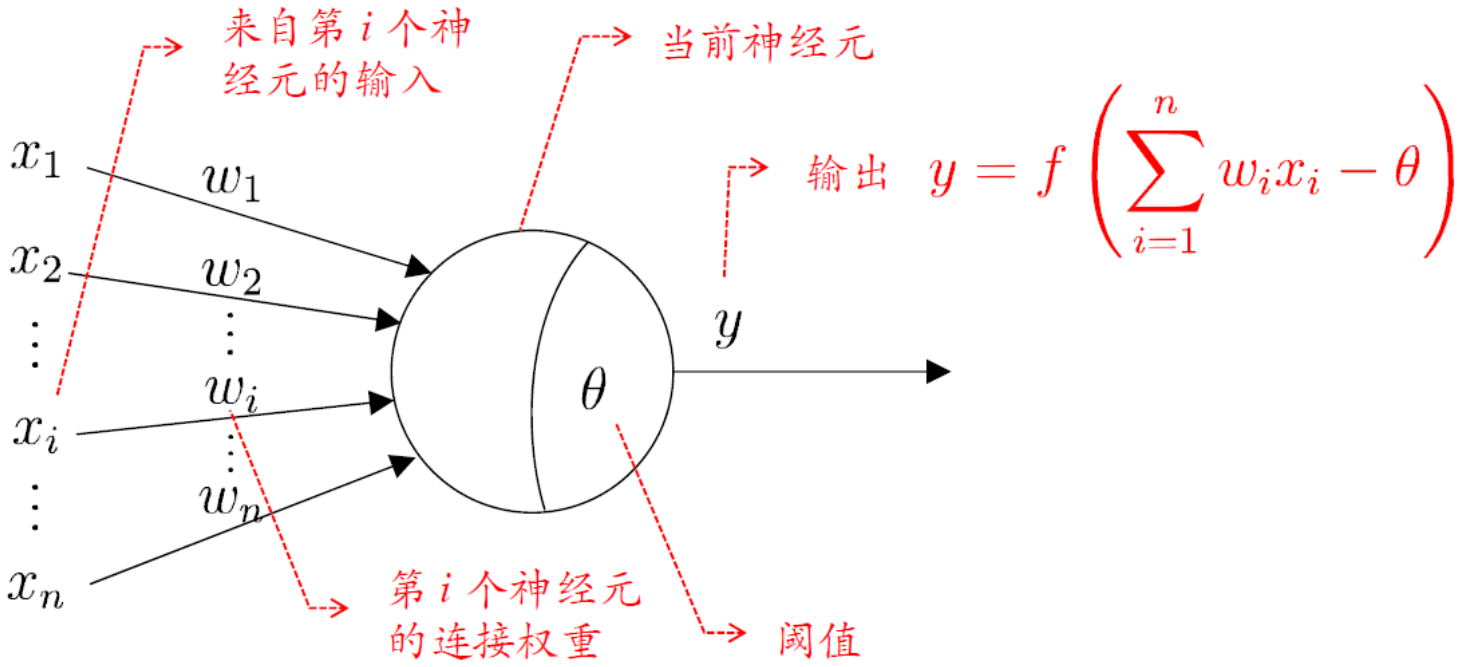
单个神经元的结构


Dendrites
Cell body
Axon
Axonal arborization
(a)


Bias
x_1
x_2
x_m
w_{k1}
w_{k2}
w_{km}
Inputs
Σ
v_k
Activation function
$\varphi(\cdot)$
Output
Summing junction
Synaptic weights (including bias)
(b)

M-P 神经元模型[McCulloch and Pitts, 1943]

- **输入**：来自其他 n 个神经云传递过来的输入信号。
- **处理**：输入信号通过带权重的连接进行传递，神经元接受到总输入值将与神经元的阈值进行比较。
- **输出**：通过激活函数的处理以得到输出。



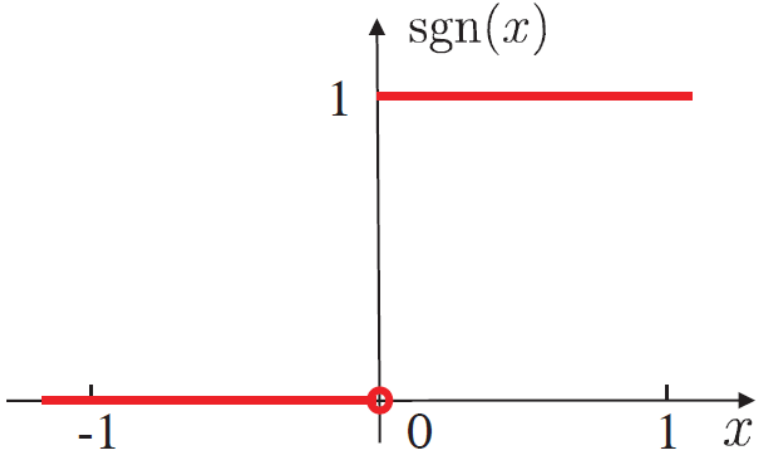
中山大學 SUN YAT-SEN UNIVERSITY

3. 不同类型的ANN

- 神经元模型
- 激活函数
- 感知机模型
- 多层前馈神经网络

激活函数1：阶跃函数

- **理想激活函数**是**阶跃函数**，
  - 0：抑制神经元，1：激活神经元；
  - 不连续、不光滑（不好的性质）



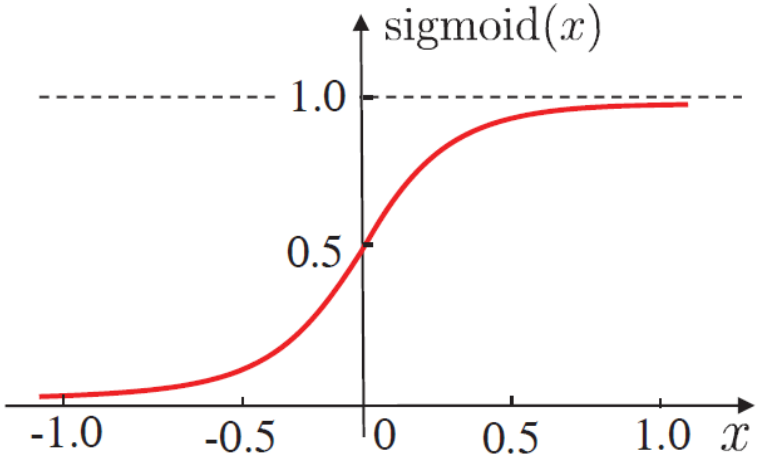
$$\mathrm{sgn}(x) = \begin{cases} 1, & \text{if } x \geq 0; \\ 0, & \text{if } x < 0. \end{cases}$$

(a) 阶跃函数

中山大學 SUN YAT-SEN UNIVERSITY

激活函数2：Sigmoid 函数

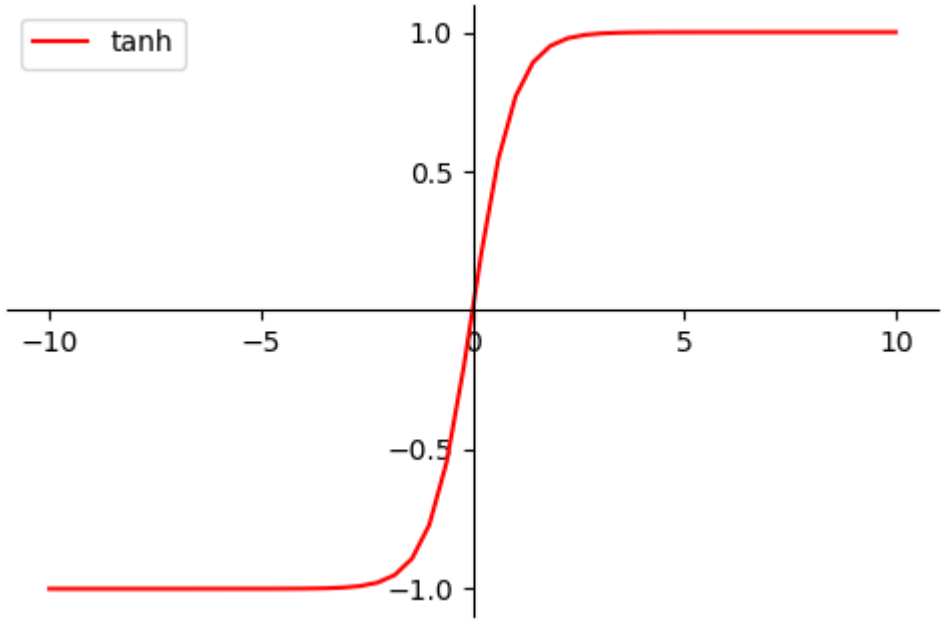
- **常用激活函数**是 **Sigmoid 函数 。**
- **优点：有较好的解释性**
- **缺点：Sigmoid函数饱和使梯度消失。sigmoid神经元的激活在接近极大值或极小值处时会饱和，在这些区域，梯度几乎为0。且其输出不是零中心的，这一情况将影响梯度下降的运作。**



$$\text{sigmoid}(x) = \frac{1}{1+e^{-x}}$$

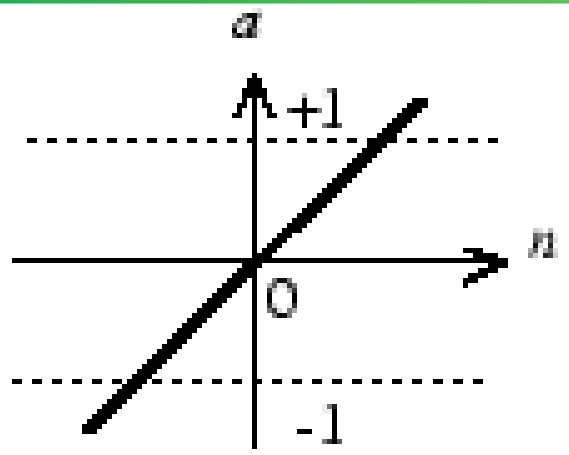
激活函数3： Tanh或Tansig 函数

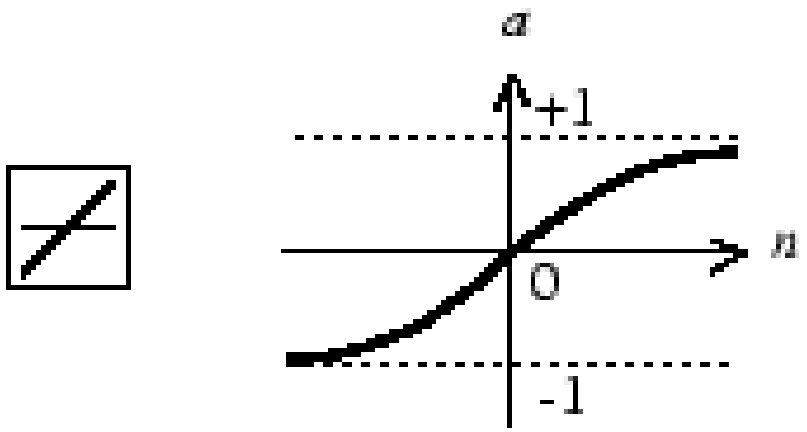
- **优点**：Sigmoid 函数向下做了平移；输出是零中心的，值的范围在（-1，1）之间
- **缺点**：和Sigmoid函数一样，饱和使梯度消失。计算缓慢。



$$y = \text{tansig}\,(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

Matlab中激活（传递）函数





$a = purelin(n)$

Linear Transfer Function

$a = tansig(n)$

Tan-Sigmoid Transfer Function

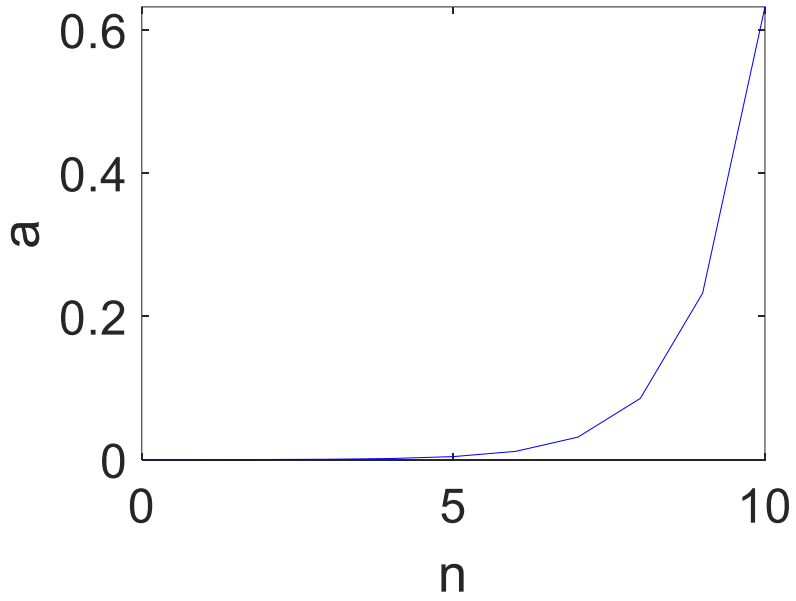
a = purelin(n) = n

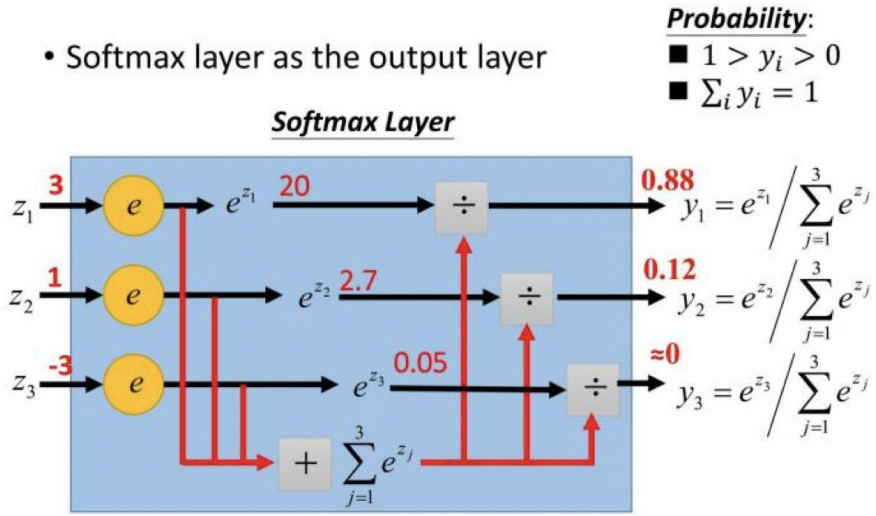
直接输出函数

a = tansig(N)
= 2/(1+exp(-2*N))-1

tanh函数

Matlab中激活（传递）函数





a = softmax(n) = exp(n)/sum(exp(n))

softmax函数

- **用于输出某个类别概率**
- **用于多分类网络的输出**

3. 不同类型的ANN

- 神经元模型
- 激活函数
- **感知机模型**
- 多层前馈神经网络

感知机

单层感知机（输入层不算做1层）

- **结构**：两层神经元（3个神经元）组成，
  - **输入层**接受外界输入信号传递给输出层，
  - **输出层**是M-P神经元（阈值逻辑单元）
- **功能**：能够容易地**实现逻辑与、或、非运算**

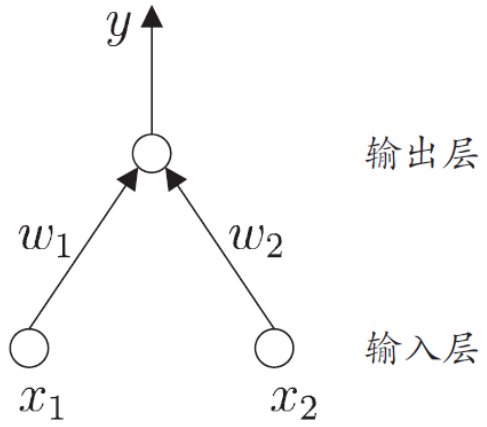


图 5.3　两个输入神经元的感知机网络结构示意图

感知机的例子

感知机能够容易地实现逻辑与、或、非运算

1) $x_1 \wedge x_2$ "与" $w_1 = w_2 = 1, \theta = 0$，则 $y = f(1 \cdot x_1 + 1 \cdot x_2 - 2)$

仅在 $x_1 = x_2 = 1$ 时，$y = 1$.

2) $x_1 \vee x_2$ "或" $w_1 = w_2 = 1, \theta = 0.5$，则 $y = f(1 \cdot x_1 + 1 \cdot x_2 - 0.5)$

当 $x_1 = 1$ 或者 $x_2 = 1$ 时，$y = 1$.

3) $\neg x_1$ "非" $w_1 = -0.6, w_2 = 0, \theta = -0.5$，则

当 $x_1 = 1$ 时，$y = 0$；当 $x_1 = 0$ 时，$y = 1$.

感知机求解与、或、非问题

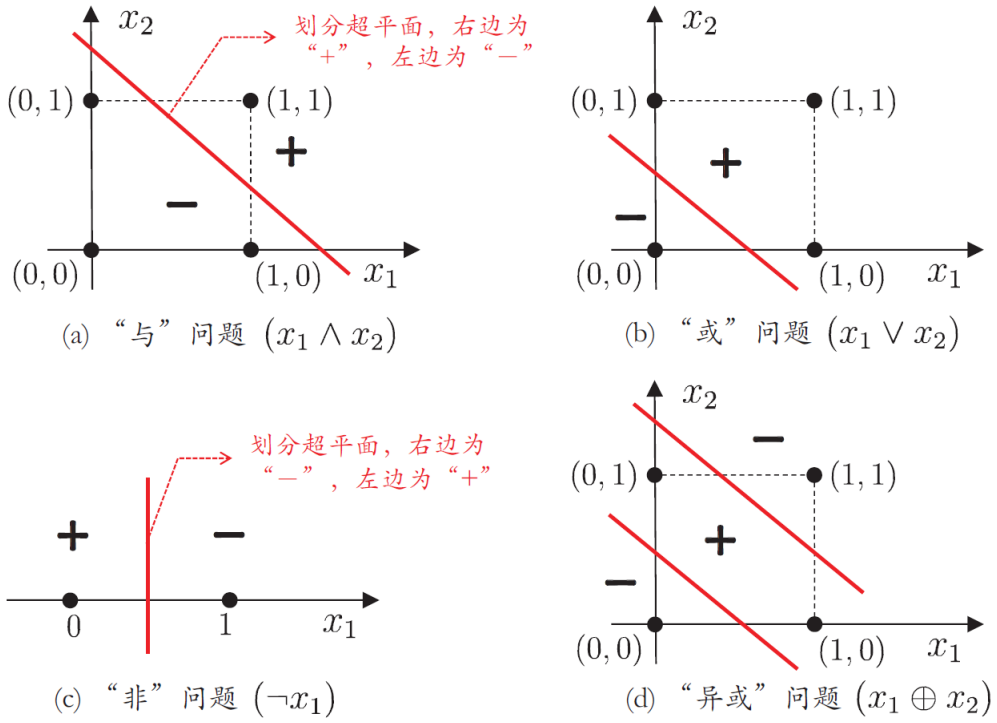


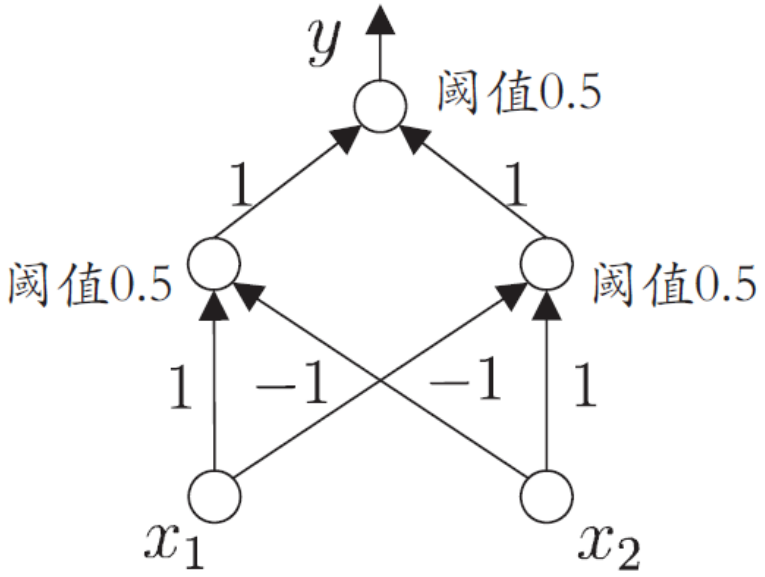
图 5.4　线性可分的“与”“或”“非”问题与非线性可分的“异或”问题

感知机学习能力

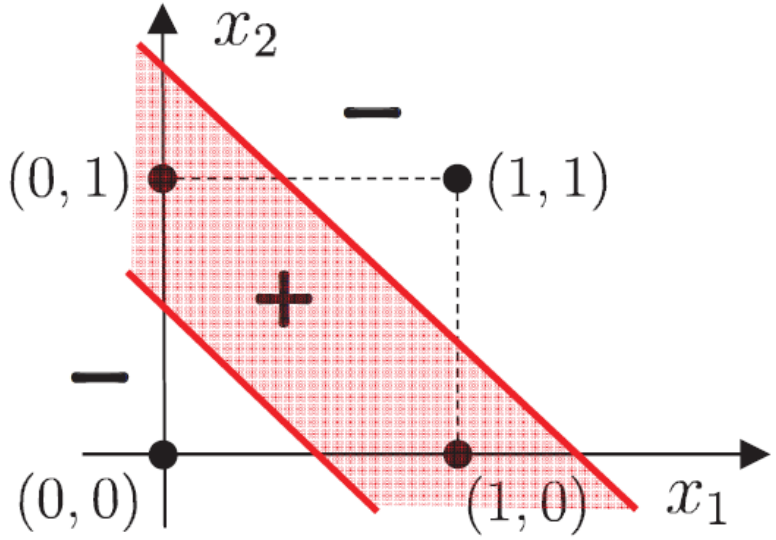
- 若两类模式**线性可分**，则感知机的学习过程一定会**收敛**；否则感知机的学习过程将会发生震荡 [Minsky and Papert, 1969]
- **单层感知机的学习能力非常有限，只能解决线性可分问题**
- 事实上，**与、或、非问题是线性可分的**，因此感知机学习过程能够求得适当的权值向量.
- 而异或问题不是线性可分的，感知机不能求得合适解
- 对于非线性可分问题，如何求解？ **多层感知机**

解决异或问题的两层感知机

- 输出层与输入层之间的一层神经元，被称之为**隐层或隐含层**，
- 隐含层和输出层神经元都是具有激活函数的功能神经元



(a) 网络结构



(b) 分类区域

图 5.5　能解决异或问题的两层感知机

感知机学习：求解算法

给定训练数据集，权重 $w_i(i=1,2,\cdots,n)$ 与阈值 θ 可以通过学习得到**感知机学习规则**

对训练样例 (x,y)，若当前感知机的输出为 $\hat{y}$，则**感知机权重调整规则**为：

$$w_i \leftarrow w_i + \triangle w_i$$
$$\triangle w_i = \eta(y-\hat{y})x_i$$

其中 $\eta \in (0,1)$ 称为学习率

若感知机对训练样例 (x,y) 预测正确，则感知机不发生变化；

否则根据错误程度进行权重的调整

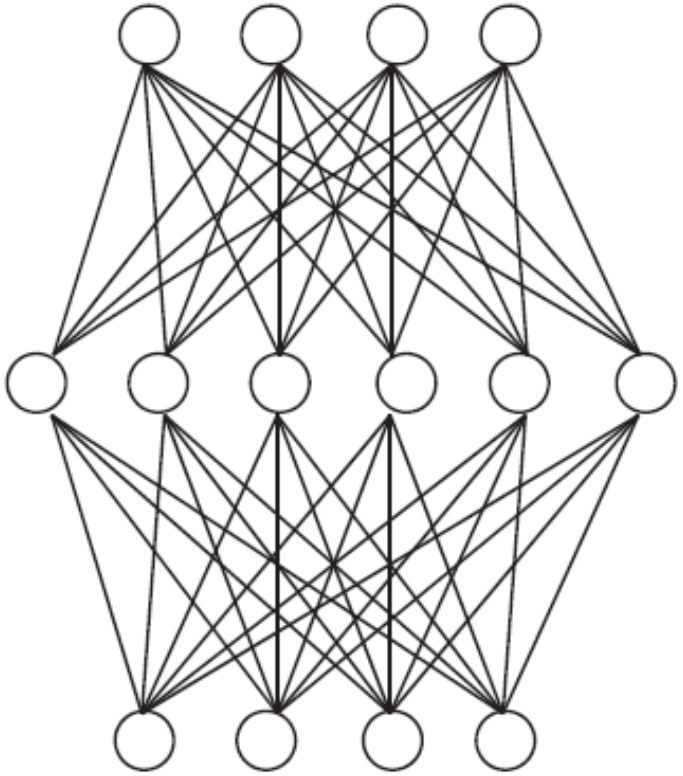
3. 不同类型的ANN

- 神经元模型
- 激活函数
- 感知机模型
- 多层前馈神经网络

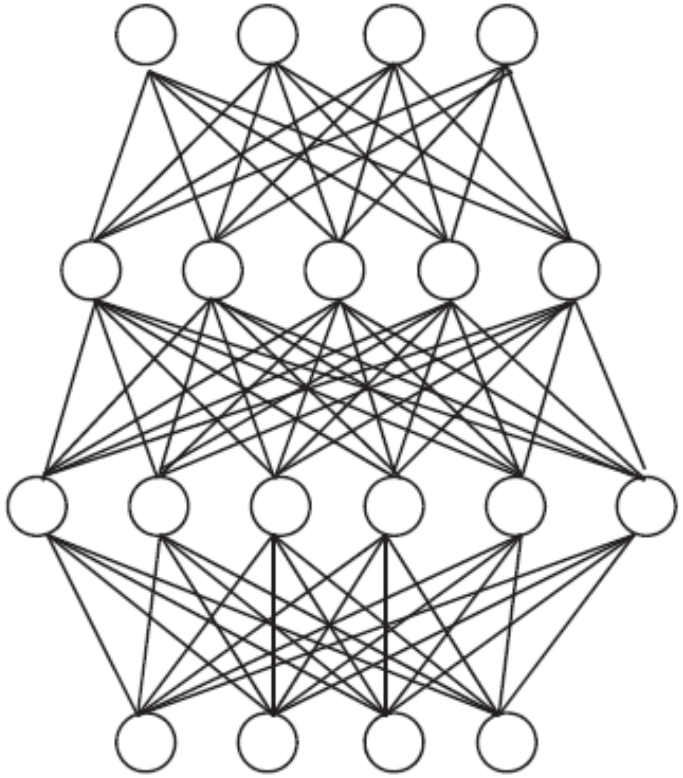
多层前馈神经网络(包含隐含层)

- **中文**：多层前馈神经网络（多层前向神经网络）或多层感知机或BP神经网络
- **英文**：multilayer feedforward neural network或Multilayer Perceptron或Back Propagation neural network
- **定义**：每层神经元与下一层神经元全互联，神经元之间不存在同层连接也不存在跨层连接
- **前馈**：输入层接受外界输入，隐含层与输出层神经元对信号进行加工，最终结果由输出层神经元输出
- **学习**：根据训练数据来调整神经元之间的“连接权”以及每个功能神经元的“阈值”

多层前馈网络结构



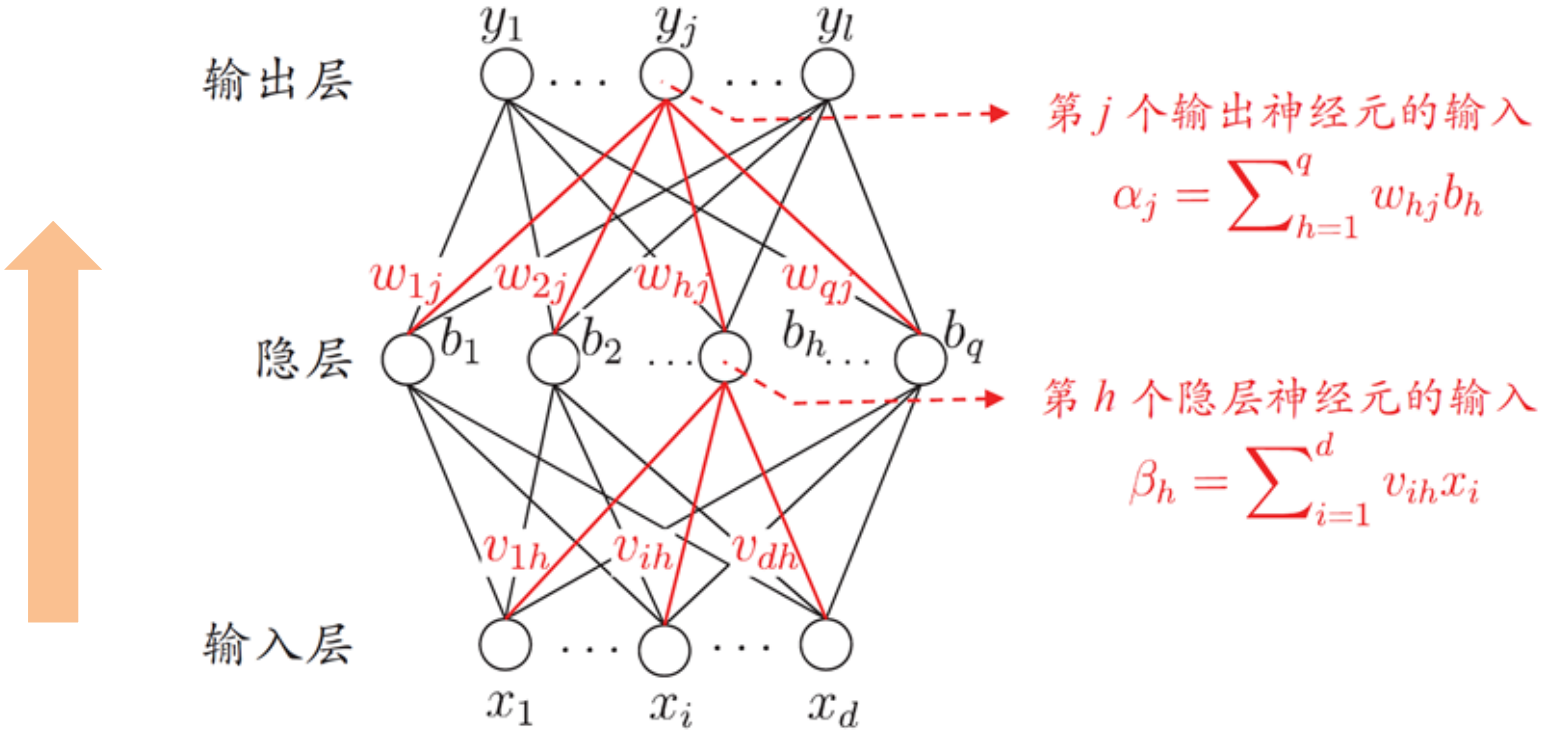
(a) 单隐层前馈网络



(b) 双隐层前馈网络

多层前馈网络结构

分层结构： 给定一个拥有 d个输入神经元，l个输出神经元，q个隐层神经元的多层前向前馈网络结构.



θ_j 输出层第j 个神经元阈值；γ_h 隐含层第h 个神经元阈值；v_{ih} 输入层与隐层神经元之间的连接权重； w_{hj}隐层与输出层神经元之间的连接权重；

因此网络中需要$(d+l+1)q+l$个参数需要优化

$$(i=1,\cdots,d,h=1,,\cdots,q,j=1,\cdots,l)$$

中山大學 SUN YAT-SEN UNIVERSITY

多层前馈网络的特点

优点（表示能力）

- **只需要一个包含足够多神经元的隐层，多层前馈神经网络就能以任意精度逼近任意复杂度的连续函数 [Hornik et al.，1989]**

局限

- **ANN由于强大表示能力，经常遭遇过拟合. 表现为：训练误差持续降低，但测试误差却可能上升**
- **如何设置隐层神经元的个数仍然是个未决问题. 实际应用中通常使用“试错法”调整**

误差反向传播算法
(误差逆传播算法)

误差逆传播算法

中文：误差逆传播算法或者误差反向传播算法

英文：Error BackPropagation, 简称BP

特点：最成功的训练多层前馈神经网络的学习算法. BP是一个迭代优化算法,

1）优化目标：损失函数最小

2）决策变量：待求参数

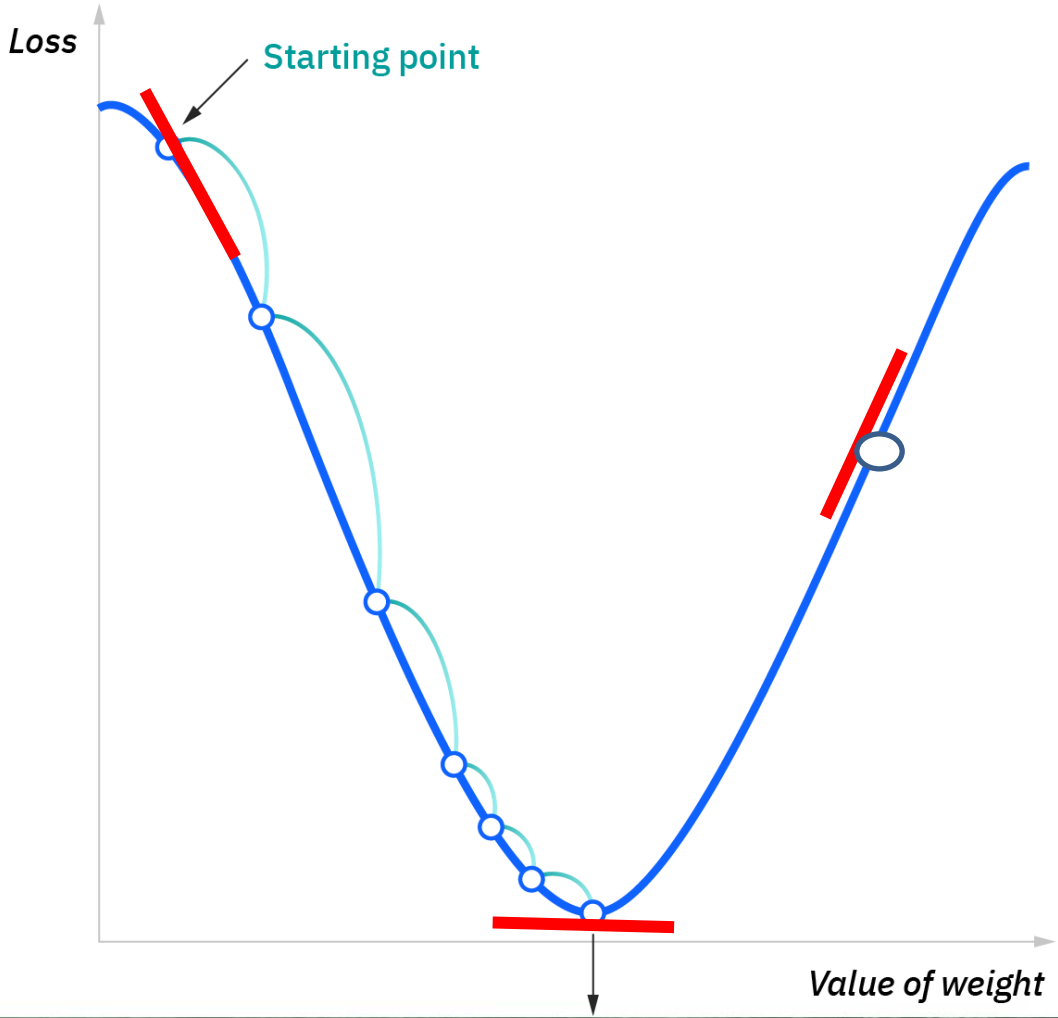
3）迭代策略：在迭代的每一轮中采用广义的感知机学习规则对参数进行更新估计, 任意的参数 的更新估计式为

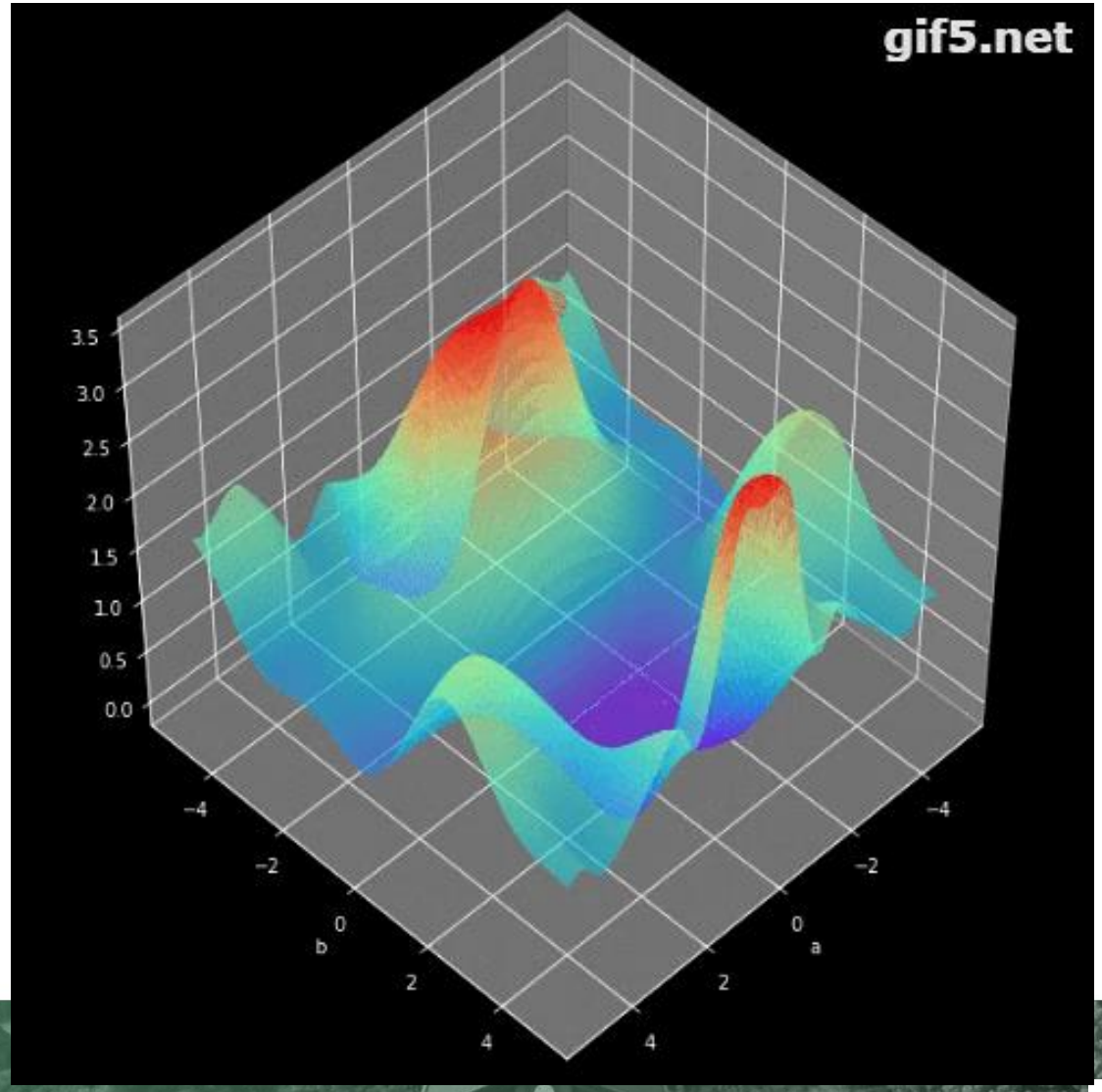
$$v \leftarrow v + \triangle v.$$

理解误差逆传播算法的几个关键

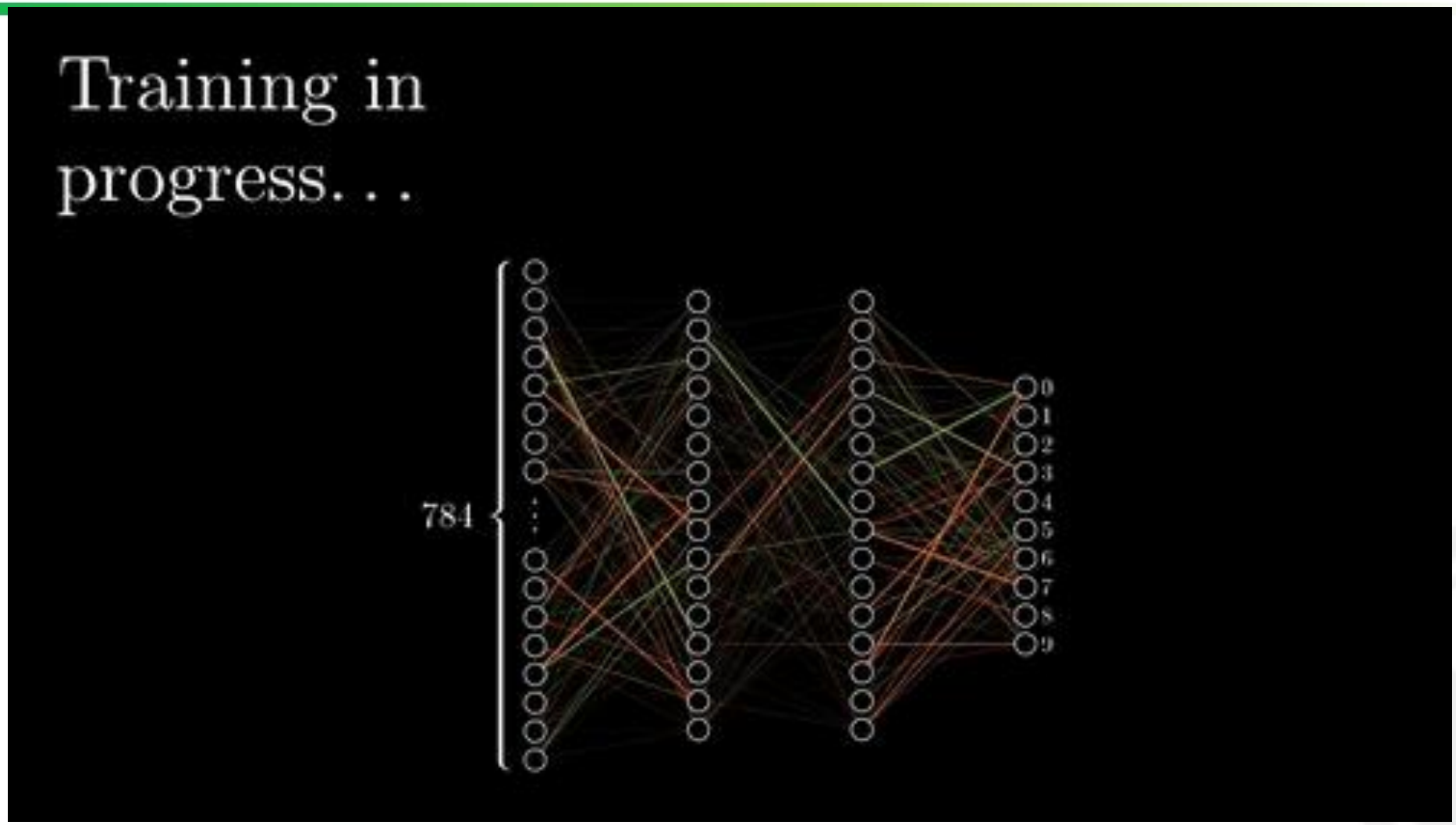
- **损失函数怎么计算(方向)？回归(误差平方和)和分类（总错误率）**
- **损失函数最小，对应的决策变量是（参数？）和其他常数（原始数据观测值）？**
- **参数有哪几类（权重和偏置）？多少个？参数求解是否有统一方法?**
- **参数求解怎么通过迭代方法获得？➔1）初始值？ 2）新的参数怎么通过旧的参数迭代？➔损失函数对参数求偏导和参数迭代方向有无关系？**
- **参数求解是唯一的吗？是全局最优的吗?**

损失函数和单个参数关系



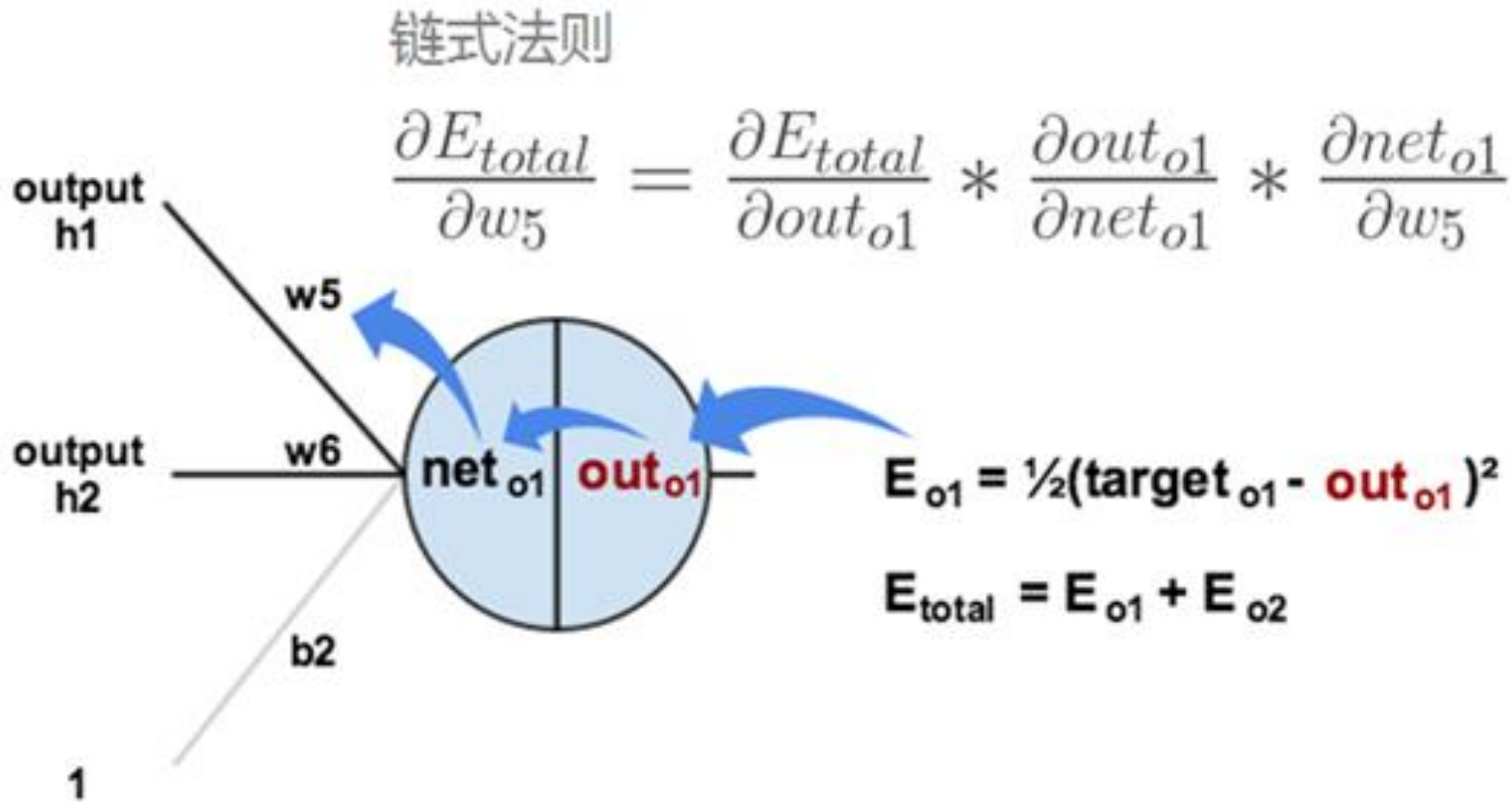

gif5.net
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
−4
−2
0
b
2
4
−4
−2
0
a
2
4

前向和后向



http://slazebni.cs.illinois.edu/fall18/assignment2.html

链式法则



根据误差结果，利用著名的“链式法则”求偏导，使结果误差反向传播从而得出权重w调整的梯度。此图是输出结果到隐含层的反向传播过程（隐含层到输入层的反向传播过程也是类似）

给定样本和参数，网络前向计算

- 对于单个样例（x_k, y_k），假设网络的实际输出为

损失函数 $E_k = \frac{1}{2}\sum_{j=1}^{l}(\hat{y}_j^k - y_j^k)^2$

y估计值 y观测值

$$\hat{y}_j^k = f(\alpha_j - \theta_j), \alpha_h = \sum_{i=1}^{q} w_{hj} b_h \quad (5.3)$$

偏置 权重

$$b_h = f(\beta_h - \gamma_h), \beta_h = \sum_{i=1}^{d} v_{ih} x_i$$

x观测值

梯度下降策略

BP算法经常特指梯度下降策略，以目标的负梯度方向对参数进行调整.

对误差 E_k ，给定学习率 η

$$\triangle w_{hj} = -\eta \frac{\partial E_k}{\partial w_{jk}}$$

$$\frac{\partial E_k}{\partial w_{hj}} = \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial_{hj}}$$

$$\begin{aligned} g_j &= -\frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \\ &= -(\hat{y}_j^k - y_j^k) f'(\beta_j - \theta_j) \\ &= \hat{y}_j^k (1 - \hat{y}_j^k)(y_j^k - \hat{y}_j^k) \quad (5.10) \end{aligned}$$

$$\triangle w_{hj} = \eta b_j g_h \quad (5.11)$$

梯度下降策略

类似的可以推导出：

$$\triangle\theta_j = -\eta g_j, \quad (5.12)$$

$$\triangle v_{ih} = \eta e_h x_i, \quad (5.13)$$

$$\triangle\gamma_h = -\eta e_h, \quad (5.14)$$

其中

$$e_h = -\frac{\partial E_k}{\partial b_h}\cdot\frac{\partial b_h}{\partial \alpha_h} = \sum_{j=1}^{h} w_{hj} g_j f'(\alpha_h - \gamma_h)$$

$$= -\sum_{j=1}^{l}\frac{\partial E_k}{\partial \beta_j}\partial b_h f'(\alpha_h - \gamma_h) = b_h(1-b_h)\sum_{j=1}^{h} w_{hj} g_j. \quad (5.15)$$

学习率 η 控制着算法每一轮迭代中的更新步长，若太长则让容易震荡，太小则收敛速度又会过慢.

$$\eta \in (0, 1)$$

BP 学习算法的步骤

输入： 训练集 $D = \{(\boldsymbol{x}_k, \boldsymbol{y}_k)\}_{k=1}^{m}$;
学习率 η.

过程：

1: 在$(0, 1)$范围内随机初始化网络中所有连接权和阈值
2: **repeat**
3: **for all** $(\boldsymbol{x}_k, \boldsymbol{y}_k) \in D$ **do**
4: 根据当前参数和式(5.3) 计算当前样本的输出 $\hat{\boldsymbol{y}}_k$;
5: 根据式(5.10) 计算输出层神经元的梯度项 g_j;
6: 根据式(5.15) 计算隐层神经元的梯度项 e_h;
7: 根据式(5.11)-(5.14) 更新连接权 w_{hj}, v_{ih} 与阈值 θ_j, γ_h
8: **end for**
9: **until** 达到停止条件

输出： 连接权与阈值确定的多层前馈神经网络

图 5.8　误差逆传播算法

标准 BP 算法 VS 累计 BP 算法

标准 BP 算法

- 每次针对单个训练样例更新权值与阈值.
- 参数更新频繁, 不同样例可能抵消, 需要多次迭代.

累计 BP 算法

$$E = \frac{1}{m}\sum_{k=1}^{m} E_k$$

- 其优化的目标是最小化整个训练集上的累计误差
- 读取整个训练集一遍才对参数进行更新, 参数更新频率较低.

实际应用

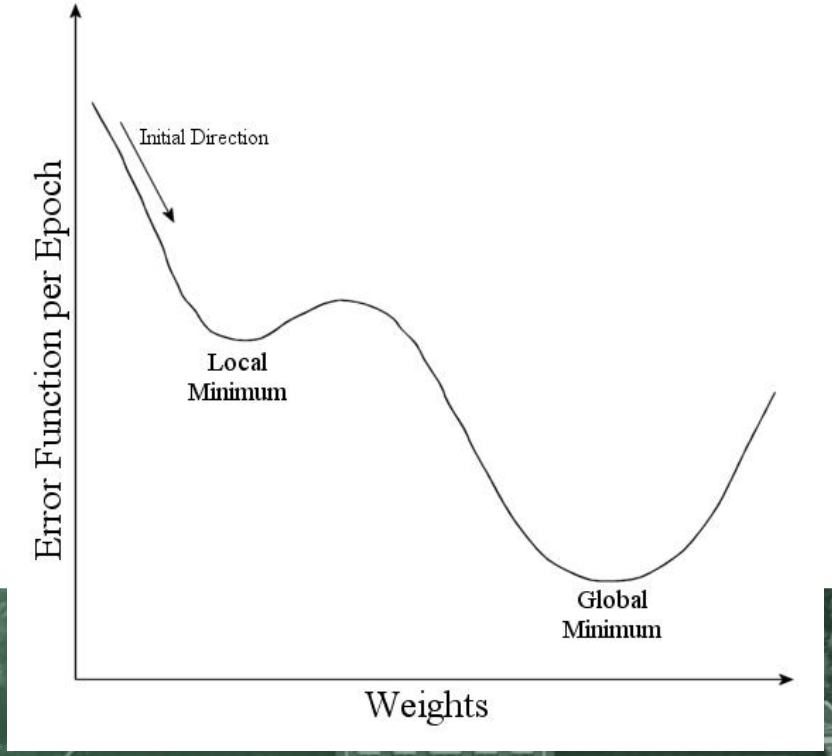
但在很多任务中, 累计误差下降到一定程度后, 进一步下降会非常缓慢, 这时标准BP算法往往会获得较好的解, 尤其当训练集非常大时效果更明显.

误差逆传播算法的问题

- **问题：BP算法会导致神经网络经常遭遇过拟合. 表现为：训练误差持续降低, 但测试误差却可能上升**
- **缓解过拟合的策略**
  - **早停：在训练过程中, 若训练误差降低, 但验证误差升高, 则停止训练**
  - **正则化：在误差目标函数中增加一项描述网络复杂程度的部分, 例如连接权值与阈值的平方和**

参数寻优方法

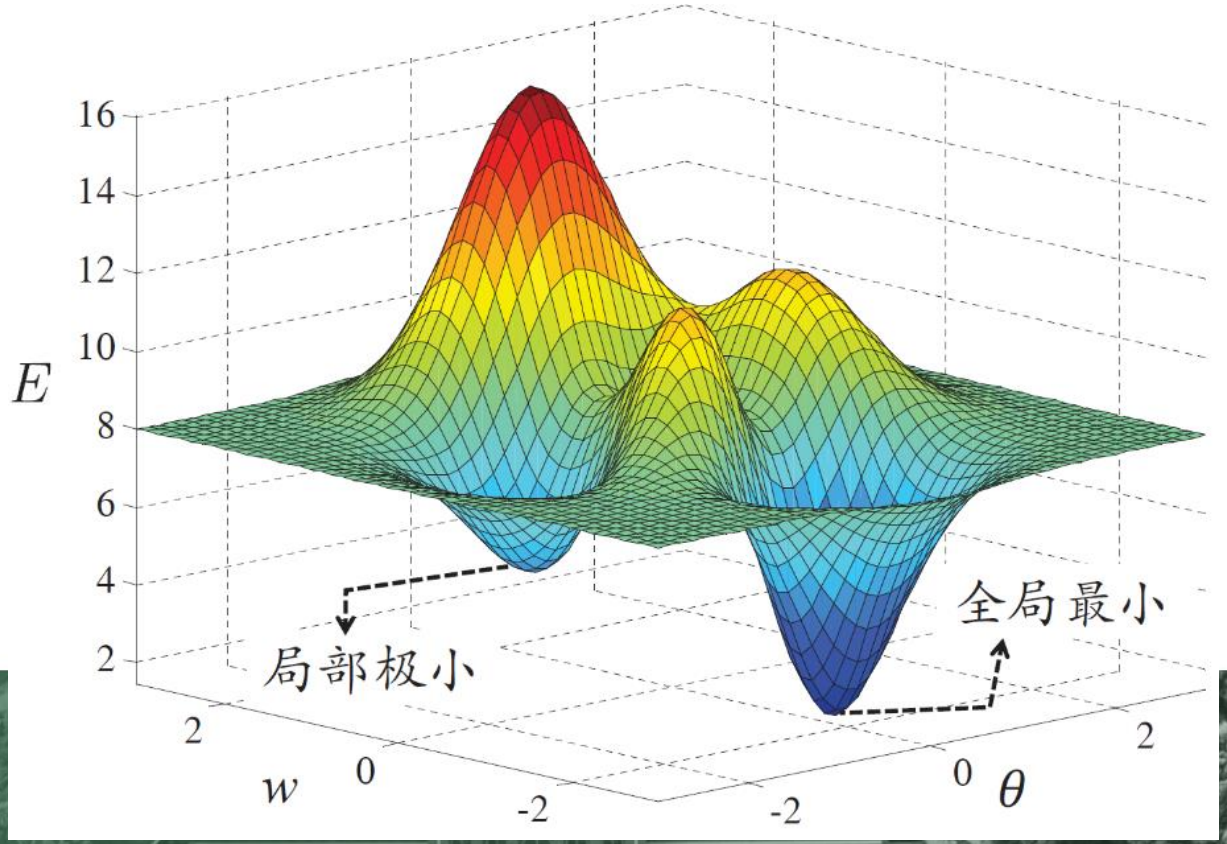
基于梯度的搜索是使用最为广泛的参数寻优方法. 如果误差函数仅有一个局部极小，那么此时找到的局部极小就是全局最小；如果误差函数具有多个局部极小，则不能保证找到的解是全局最小.



中山大學 SUN YAT-SEN UNIVERSITY

全局最小 vs 局部极小

$(w^*;\theta^*)$是**局部极小解**或者**全局最小解**. 对应的是误差函数的局部最小解和全局最小值. 显然参数空间梯度为零的点, 只要取误差函数值小于邻点的误差函数值, 就是局部极小点➔**可能存在多个局部极小值, 但却只会有一个全局极最小值**



中山大學
SUN YAT-SEN UNIVERSITY

跳出”局部最小的策略

- 在现实任务中，通常采用以下策略“跳出”局部极小，从而进一步达到全局最小。
- 多组**不同的初始参数**优化神经网络，选取误差最小的解作为最终参数➔ **Matlab中每次ANN的训练结果都不一样**。
- **模拟退火技术** [Aarts and Korst，1989]. 每一步都以一定的概率接受比当前解更差的结果，从而有助于跳出局部极小。
- **随机梯度下降**. 与标准梯度下降法精确计算梯度不同，随机梯度下降法在计算梯度时加入了随机因素。
- **遗传算法** [Goldberg，1989]. 遗传算法也常用来训练神经网络以更好地逼近全局极小。

用Matlab构建ANN

ANN建模的七个主要步骤

1. **收集数据**
2. **创建网络**
3. **配置网络**
4. **初始化权重和偏置**
5. **训练网络**
6. **验证网络（训练后的分析）**
7. **使用网络**

根据问题和网络功能选择网络类型?

选择网络结构?

选择训练算法?

确定最优网络结构解决问题

构建ANN模型基本步骤

步骤	基本命令	功能
数据预处理	minmax	数据标准化
	例如P=data(1:80,:)，Q=data(81:100,:)	将原始数据分割成练集P和验证集Q
初始化模型	net=newlin； newp； newlvq； newff； newc 或者fitnet	初始化不同ANN的结构
训练模型	net.trainParam.*=	制定特点训练参数
	Train(net,p)	训练网络
预测	y_train=Sim(net,P)	训练集预测
	y_valid=Sim(net,Q)	验证集预测
性能评估	y_train和y_valid的处理	数据后处理
	Corr；mse以及自定义性能指数	训练集和验证集的预测性能的评估

在Matlab中生成不同类型的ANN

ANN模型	旧命令	新命令	功能
简单线性模型	newlin	linearlayer	线性预测（一元或多元线性回归）
感知器模型	newp	perceptron	二分类判别； 三分类判别；
学习向量量化（LVQ）模型	newlvq	lvqnet	二分类判别； 三分类判别；
多层BPANN	newff	feedforwardnet ,fitnet, fitcnet,fitrnet	非线性预测； 多分类判别
竞争型网络	newc	competlayer	分类判别

构建不同类型的ANN

命令	输入变量	功能
newlin(P,S,ID,LR) returns a new linear layer.	P - RxQ matrix of Q representative input vectors. S - Number of elements in the output vector. ID - Input delay vector, default = [0]. LR - Learning rate, default = 0.01;	线性模型
NET = newp(P,T,TF,LF) Returns a new perceptron.	P - RxQ matrix of Q1 representative input vectors. T - SxQ matrix of Q2 representative target vectors. TF - Transfer function, default = 'hardlim'. LF - Learning function, default = 'learnp'.	线性分类
newff(P,T,S) returns an N layer feed-forward backprop network.	P - RxQ1 matrix of Q1 representative R-element input vectors. T - SNxQ2 matrix of Q2 representative SN-element target vectors. Si - Sizes of N-1 hidden layers, S1 to S(N-1), default = []. (Output layer size SN is determined from T.)	多层BPANN

在Matlab中生成多层前馈网络

新命令	所属工具箱	功能
nftool	深度学习	使用GUI生成用于回归
nprtool	深度学习	使用GUI生成用于分类
nctool	深度学习	使用GUI生成用于聚类
feedforwardnet	深度学习	回归（隐含层是 tansig；输出层是 purelin ： y=x函数）
patternnet	深度学习	分类（隐含层是 tansig 或tanh函数；输出层是softmax ）
selforgmap	深度学习	聚类
fitnet	深度学习	非线性回归；多分类判别
fitcnet	统计与机器学习	多分类判别
fitrnet	统计与机器学习	非线性回归；

MATLAB中ANN的激活函数选择?

- 大多数网络创建函数，**会自动分配激活函数；自动归一化等预处理和反归一化等后处理。**
- 创建网络后，可调整网络属性来覆盖默认处理函数。
  - net.inputs{1}.processFcns
  - net.outputs{2}.processFcns
- 几个处理函数具有可自定义其运算的参数。如下所示：
  - net.inputs{1}.processParams{i}
  - net.outputs{2}.processParams{i}

参考：https://ww2.mathworks.cn/help/deeplearning/ug/choose-neural-network-input-output-processing-functions.html
https://ww2.mathworks.cn/help/deeplearning/ug/multilayer-neural-network-architecture.html

中山大学 SUN YAT-SEN UNIVERSITY



Train函数的用法

命令形式	含义
trainedNet = train(net,X,T,Xi,Ai,EW)	训练网络（net）根据 net.trainFcn 和 net.trainParam.
[trainedNet,tr] = train(net,X,T,Xi,Ai,EW)	返回网络和训练记录
[trainedNet,tr] = train(net,X,T,Xi,Ai,EW,Name,Value)	由一个或多个 Name,Value 成对参数指定的训练网络的附加选项。

Train用来训练浅层神经网络， trainNetwork用来训练LSTM和CNN等深度学习网络。

Train函数的输入变量

[trainedNet,tr] = train(net,X,T,Xi,Ai,EW,Name,Value)

输入变量	含义
net	Input network 网络结构体
X	Network inputs 输入变量
T	Network targets 输出变量
Xi	Initial input delay conditions
Ai	Initial layer delay conditions
EW	Error weights
Name	Specify optional comma-separated pairs of Name,Value arguments.
Value	

sim函数的用法

功能：模拟神经网络

形式：[Y,Xf,Af] = sim(net,X,Xi,Ai,T)

参数	含义
net	Network
X	Network inputs
Xi	Initial input delay conditions (default = zeros)
Ai	Initial layer delay conditions (default = zeros)
T	Network targets (default = zeros)
Y	Network outputs
Xf	Final input delay conditions
Af	Final layer delay conditions

Matlab中ANN训练算法比较

Train的训练算法函数

函数	算法
trainlm	Levenberg-Marquardt
trainbr	贝叶斯正则化
trainbfg	BFGS 拟牛顿
trainrp	弹性反向传播
trainscg	量化共轭梯度
traincgb	带 Powell/Beale 重启的共轭梯度
traincgf	Fletcher-Powell 共轭梯度
traincgp	Polak-Ribiére 共轭梯度
trainoss	单步正割
traingdx	可变学习率梯度下降
traingdm	带动量的梯度下降
traingd	梯度下降

选择多层神经网络训练算法?

- 对于给定的问题，很难知道哪种训练算法最快。
- 算法选择取决于许多因素：问题复杂性、训练集中数据点的数量、网络中权重和偏置的数量、误差目标，及功能：模式识别（分类）或函数逼近（回归）。
- 对于训练多层前馈网络，任何标准数值优化算法都可用于优化性能函数，
- 几个关键算法已显示出对ANN训练有优异性能，使用网络性能关于网络权重的梯度，或者使用网络误差关于权重的 Jacobian 矩阵。

参考：https://ww2.mathworks.cn/help/deeplearning/ug/choose-a-multilayer-neural-network-training-function.html

反向传播算法

- **目标**：前馈网络的默认损失函数是均方误差 mse - 网络输出 a 和目标输出 t 之间的平均平方误差

$$F = mse = \frac{1}{N}\sum_{i=1}^{N}(e_i)^2 = \frac{1}{N}\sum_{i=1}^{N}(t_i - a_i)^2$$

- **定义**：基于梯度和 Jacobian 矩阵，对网络参数求解，最小化损失函数的计算方法，
- **特点**：该算法涉及在网络中反向执行计算。反向传播计算是使用微积分的链式法则求导的，
- **参考**：在 [HDB96] 的第 11 章（针对梯度）和第 12 章（针对 Jacobian 矩阵）

反向传播算法：梯度下降

- Traingd是梯度下降算法。
- **梯度下降算法是最简单的优化算法**。
- 它会沿着性能函数下降最快的方向（梯度的负方向）上更新网络权重和偏置。该算法的一次迭代可写为 $x_{k+1}=x_k-\alpha_k g_k$
- x_k 是当前权重和偏置的向量，
- **g_k 是当前梯度（损失函数对参数的偏导）**，
- α_k 是学习率。
- 此方程会一直迭代，直到网络收敛。

两个叫法的不妥之处

- “反向传播”在应用于ANN训练时，有时特指梯度下降算法。此处不使用该术语，
- Matlab所有ANN训练函数都涉及在网络中反向执行计算来求出梯度和 Jacobian 矩阵的过程。所以，相较于反向传播，使用具体优化算法的名称会更清楚。
- 此外，多层网络有时也称为反向传播网络。不过，反向传播方法也可应用于许多不同网络架构。
- 事实上，任何具有可微分传递函数、权重函数和净输入函数的网络的梯度和 Jacobian 矩阵都可以通过反向传播过程使用 Matlab来计算。

选择多层神经网络训练算法?

- **中小规模回归问题：trainlm 或 trainbr**
- **中小规模分类问题：trainrp**
- **大规模回归/分类问题：trainscg**
- **需要慢训练问题：traingdx**

参考：https://ww2.mathworks.cn/help/deeplearning/ug/choose-a-multilayer-neural-network-training-function.html

Train的训练算法比较

算法	特点	备注
trainlm	通常是最快的训练函数，是多层前馈网络的默认训练函数。 对函数拟合（非线性回归）的性能优于模式识别。	这两种方法对于大型网络（具有数千个权重）往往效率较低，因为它们需要更多内存和更多计算时间
trainbfg	拟牛顿法也相当快。	
trainscg 和 trainrp	在训练大型网络和模式识别网络时，是不错的选择。其内存要求相对较小，但比标准梯度下降算法要快得多。	

训练算法的比较: trainlm

- 一般情况下，在函数逼近（回归）问题上，对于包含多达几百个权重的网络，Levenberg-Marquardt 算法具有最快的收敛。
- 如果需要非常准确的训练，这种优势尤其明显。
- 在许多情况下，能够获得比任何其他被测算法更低的均方误差。然而，随着网络中权重数量的增加，优势会减小。
- 在模式识别（分类）问题上的表现相对较差。
- 存储要求高于被测的其他算法。

训练算法: trainrp和trainscg

- trainrp 函数是在模式识别问题上最快的算法。然而，它在函数逼近问题上表现不佳。随着误差目标的降低，其性能也会下降。与考虑的其他算法相比，该算法的内存要求相对较小。
- 共轭梯度（ SCG ）算法，特别是 trainscg，似乎在各种各样的问题上都表现良好，特别是对于具有大量权重个数的网络。SCG 在函数逼近问题上几乎与 LM 一样快（对于大型网络更快），在模式识别问题上几乎与 trainrp 一样快。
- 当误差减小时， SCG性能不会像 trainrp 性能下降得那样快。SCG算法具有相对适中的内存要求。

训练算法: trainbfg和traingdx

- trainbfg 的表现与 trainlm 相似。它不需要像 trainlm 那样多的存储空间，但所需的计算量确实会随着网络规模的增大而呈几何增长，因为每次迭代都必须计算等效逆矩阵。
- 可变学习率算法 traingdx 通常比其他方法慢得多，其存储要求与 trainrp 大致相同，但它仍可用于求解某些问题。
- Traingdx可以配合早停法。在某些情况下，收敛速度越慢越好。例如，当使用早停法时，如果使用收敛过快的算法，可能会产生不一致的结果。您可能会错过在验证集上误差最小化的点。

小结

思考

- 人工神经网络怎么发展起来的?
- 什么是人工神经网络?
- 人工神经网络有哪几种常用类型?
- 每种类型的ANN的结构、功能和实现？（具体用于解决什么类型的问题？Matlab如何实现？）
- 构建ANN模型的步骤？怎么评估一个ANN模型？怎么能训练出一个好的ANN模型?

延伸

- 具有其他结构和功能的人工神经网络?
- ANN处理时序问题的特殊性?
- ANN处理图像?
- ANN处理大数据?
- ANN与并行计算?

推荐的慕课

- 机器学习 https://www.icourse163.org/learn/ZJU-1206573810
- 机器学习与人工智能https://www.icourse163.org/learn/ZJU-1206689820
- 机器学习 https://www.icourse163.org/learn/CUG-1003556007